



Neural Network

Multi-Layer ANN

Prof. Anil Singh Parihar

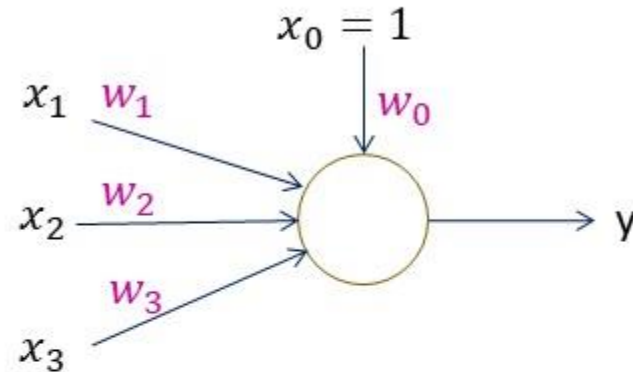
Department of Computer Science & Engineering

Delhi Technological University, Delhi

Single Neuron

$$\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\bar{w} = [w_1 w_2 w_3]$$



Activation Function: ReLu

$$\varphi(z) = \max\{0, z\}$$

$$y = \varphi(z)$$

$$\begin{aligned} \bar{w}\vec{x} + w_0 &= z \\ [w_1 w_2 w_3] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + w_0 &= z \\ y &= \varphi(z) \end{aligned}$$

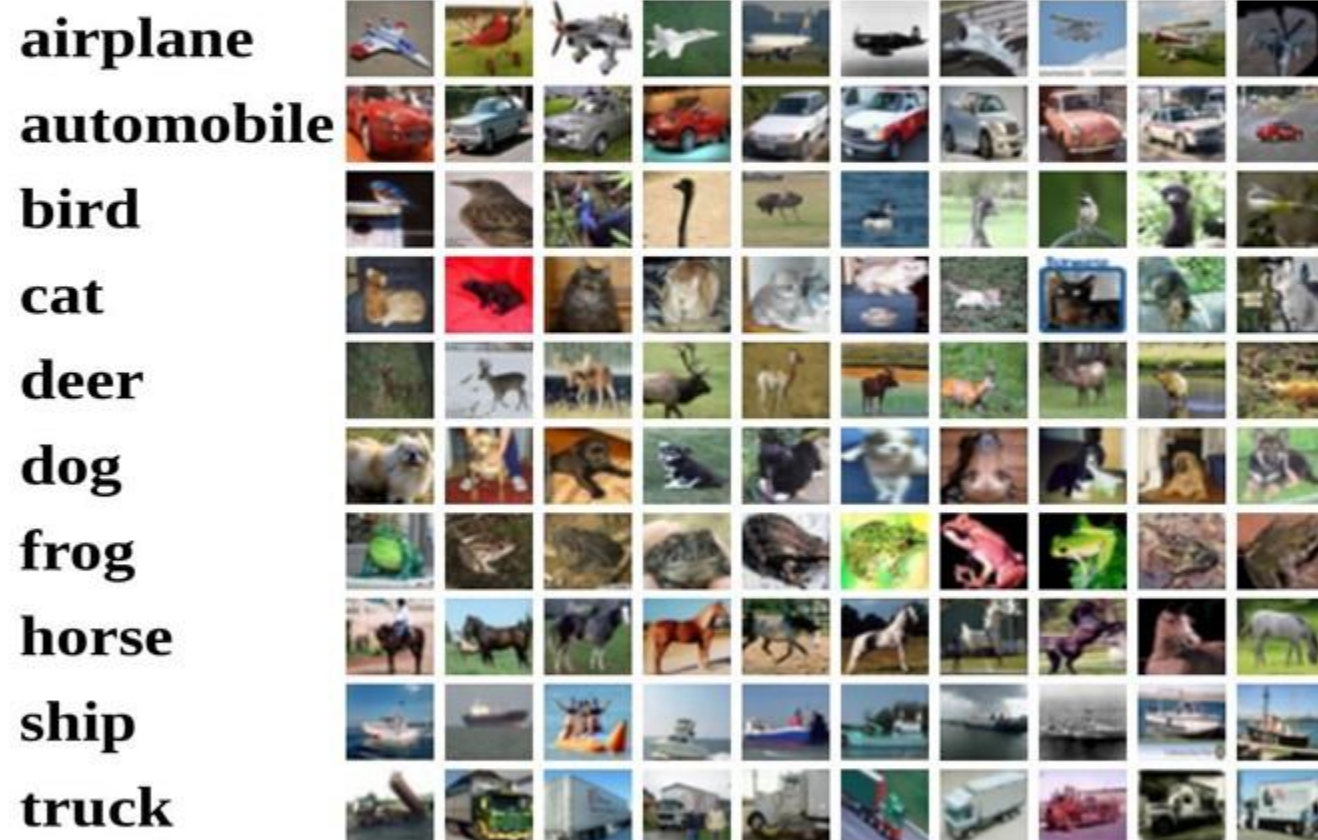
Let

$$\bar{w} = [0.9, 0, 1] \quad w_0 = -0.5 \quad \vec{x} = \begin{bmatrix} 1 \\ 0.2 \\ 0.7 \end{bmatrix}$$

$$[0.9, 0, 1] \begin{bmatrix} 1 \\ 0.2 \\ 0.7 \end{bmatrix} + (-0.5) = 1.1$$

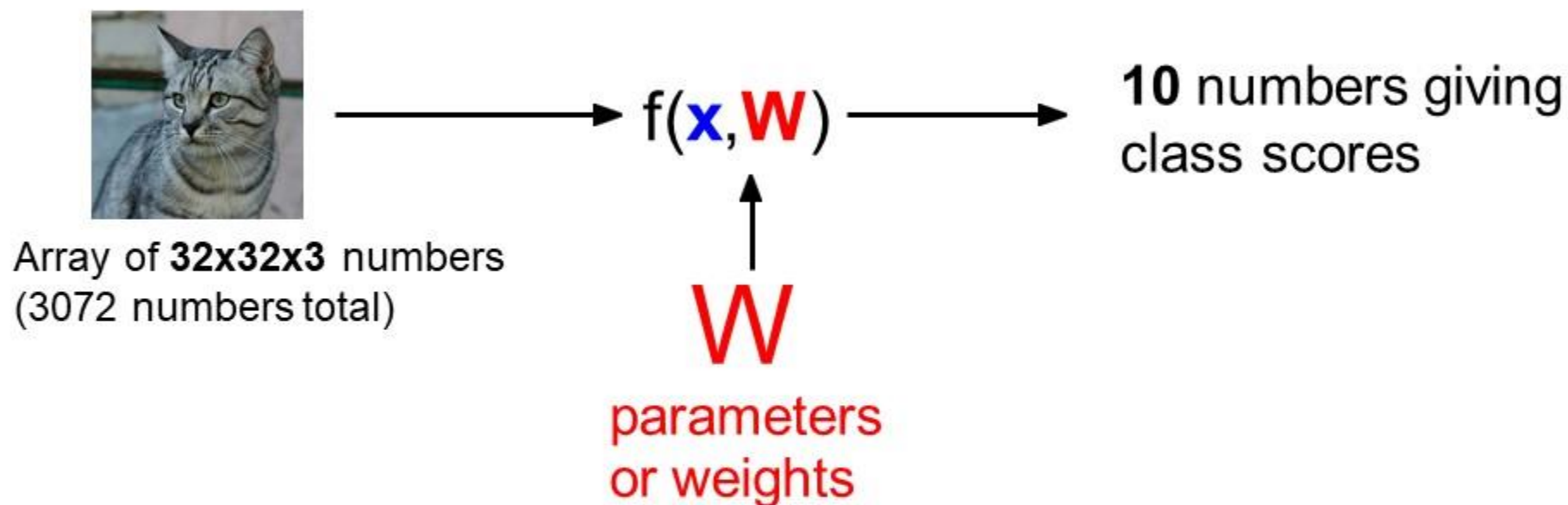
$$y = \varphi(1.1) = 1.1$$

CIFAR10 Dataset



10 classes
50,000 training images
10,000 testing images

Linear Classifier

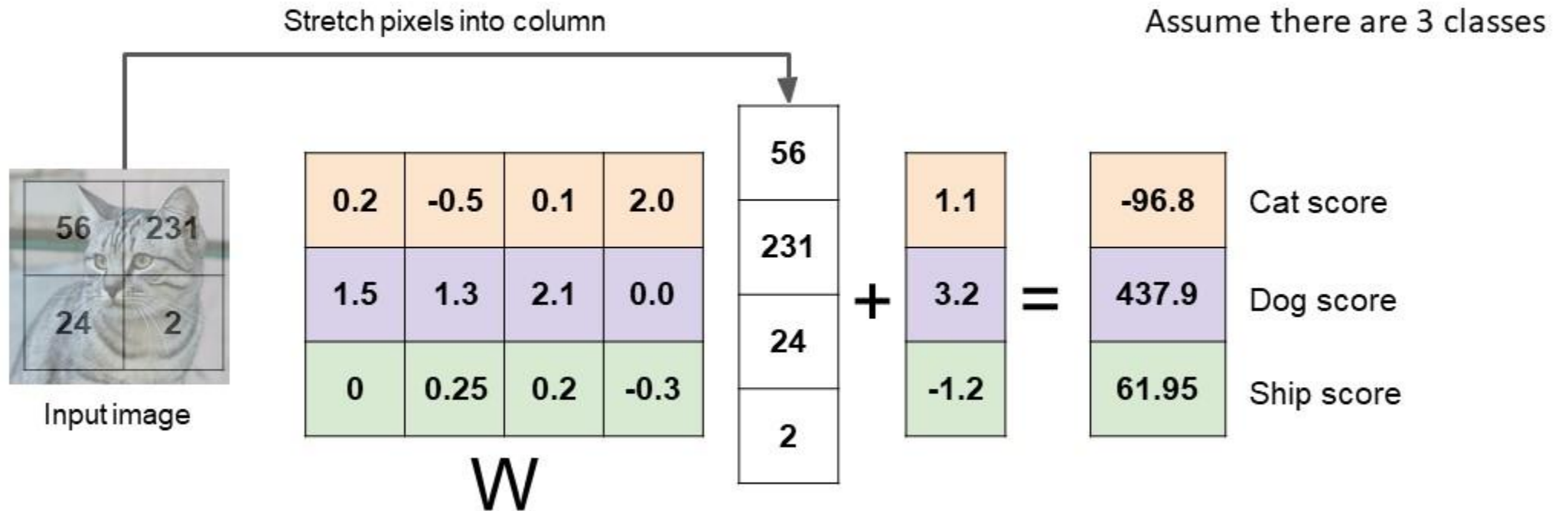


Linear function

$$\boxed{f(\mathbf{x}, \mathbf{W})} = \boxed{\mathbf{W}} \boxed{\mathbf{x}} + \boxed{W_0} \quad 10 \times 1$$

$10 \times 1 \quad 10 \times 3072 \quad 3072 \times 1$

Cat Classifier

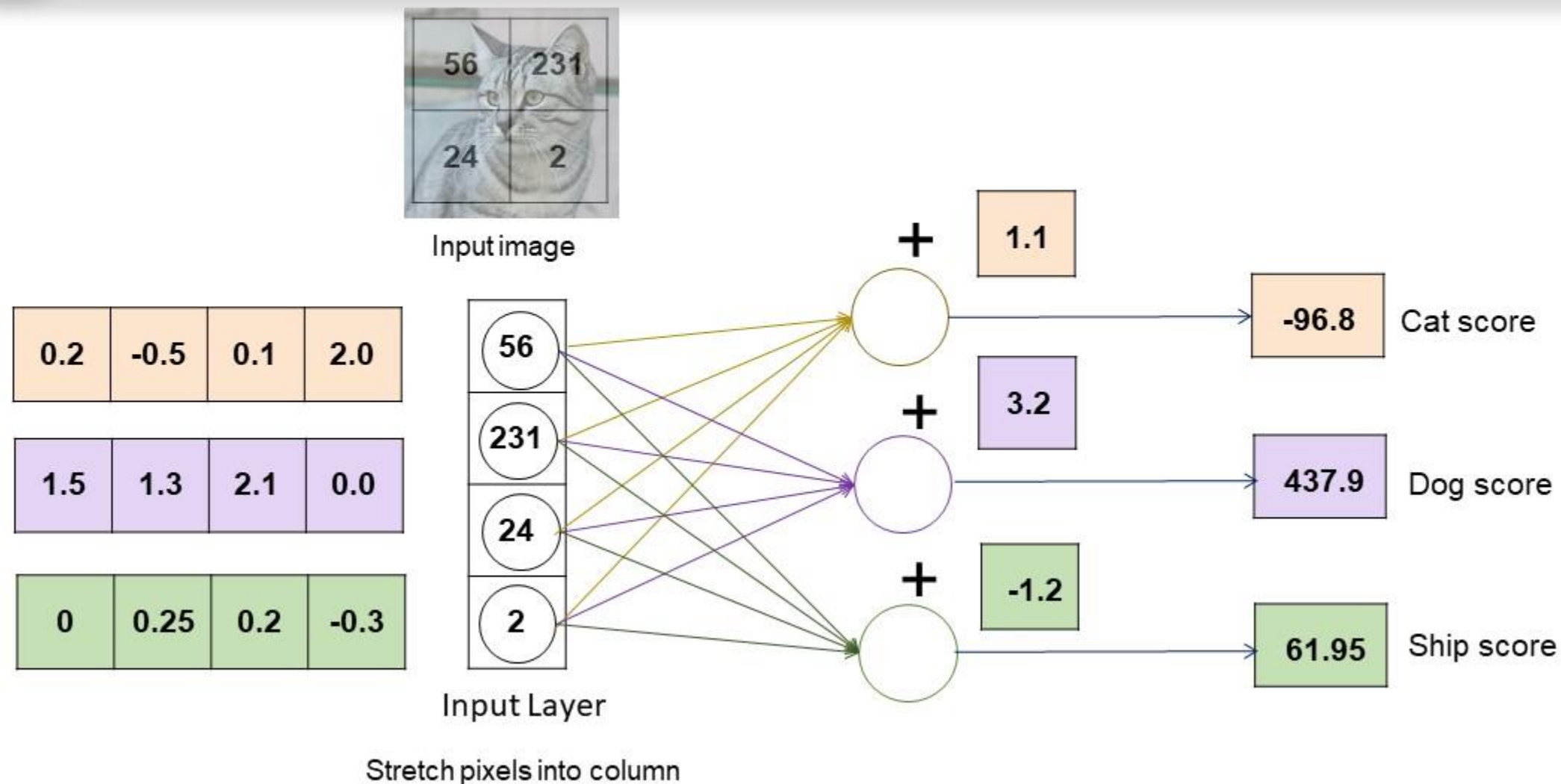


Let input image is gray scale image with only 4 pixel for the purpose of Illustration.

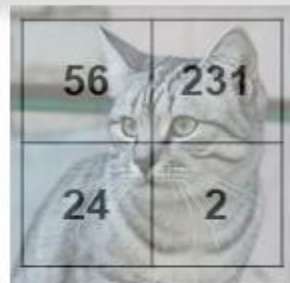
Thus, size of weight matrix:
#classes x #rows in the input
i.e. 3x4

Add bias

Cat Classifier



Cat Classifier



Input image

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3



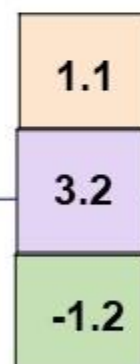
Input Layer

Stretch pixels into column

+

+

+



Cat score

Dog score

Ship score

Cat Classifier

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

56
231
24
2

1.1
3.2
-1.2

-96.8
437.9
61.95

Cat score
Dog score
Ship score

W X W_0 $f(x, W) = Wx$

3x4 4x1 3x1 3x1

Size of weight matrix:
#classes x #rows in the input
i.e. 3x4

Interpretation of Linear Classifier

airplane

automobile

bird

cat

deer

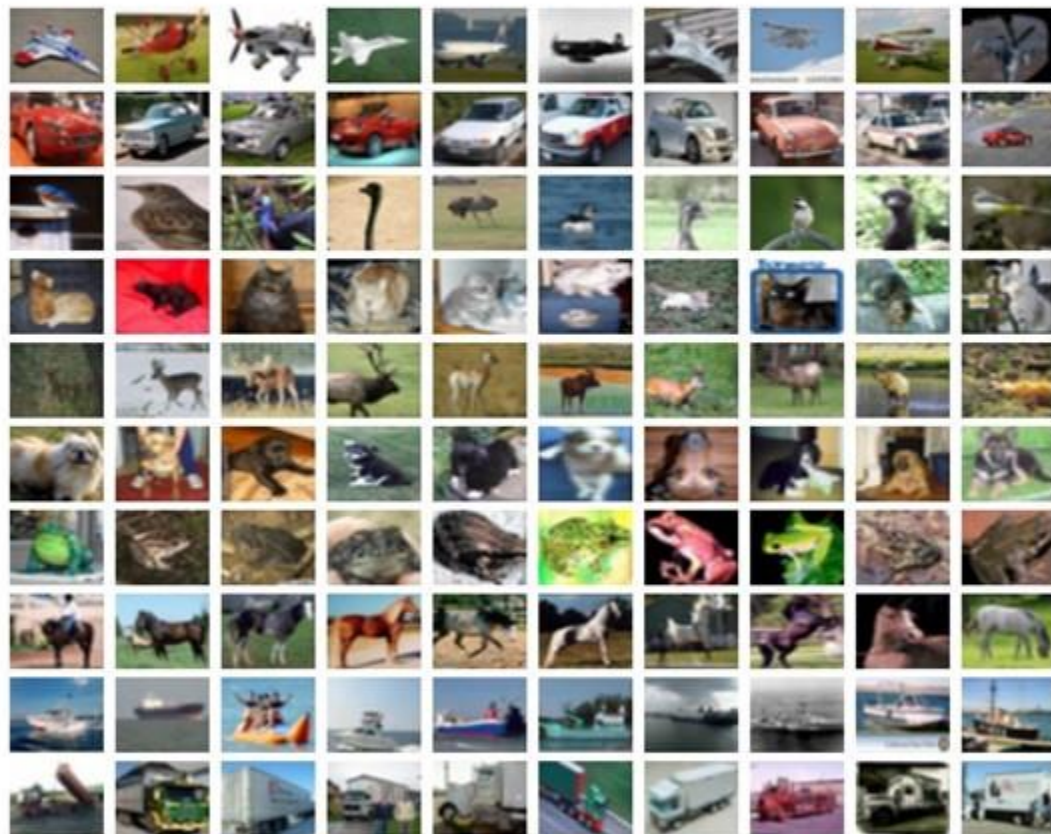
dog

frog

horse

ship

truck

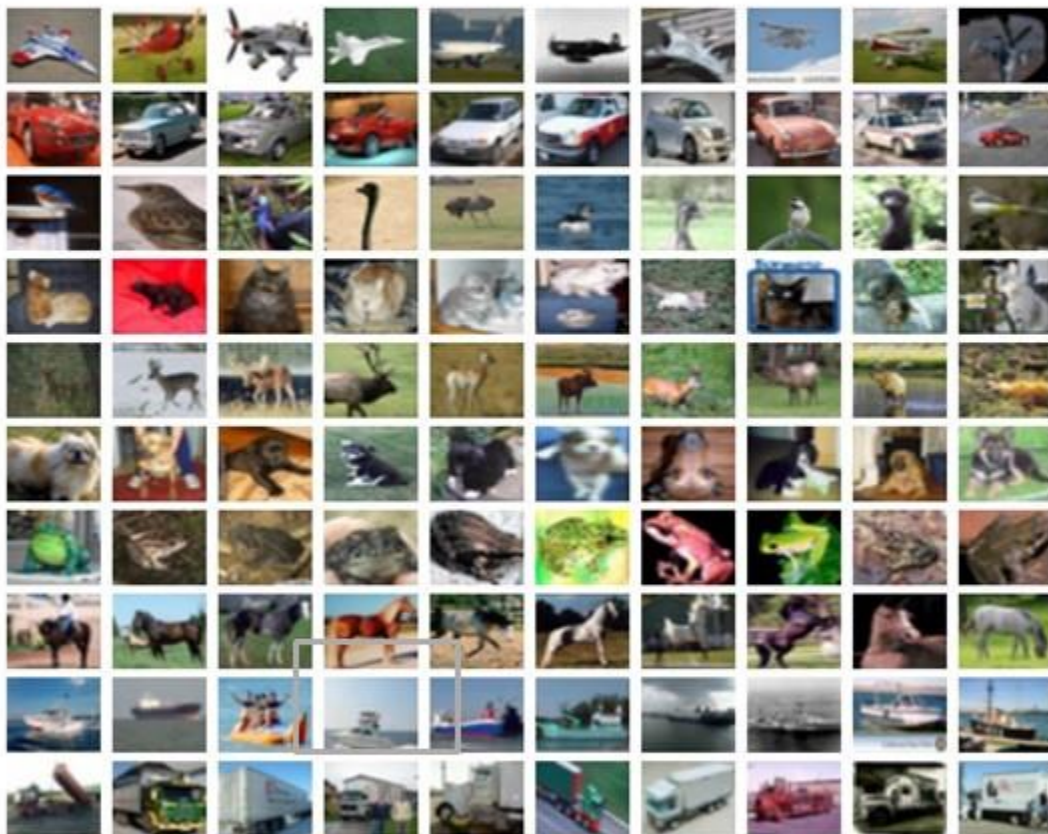


$$f(x, W) = Wx + b$$

Train a ANN with linear
activation function

Interpretation of Linear Classifier

airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck

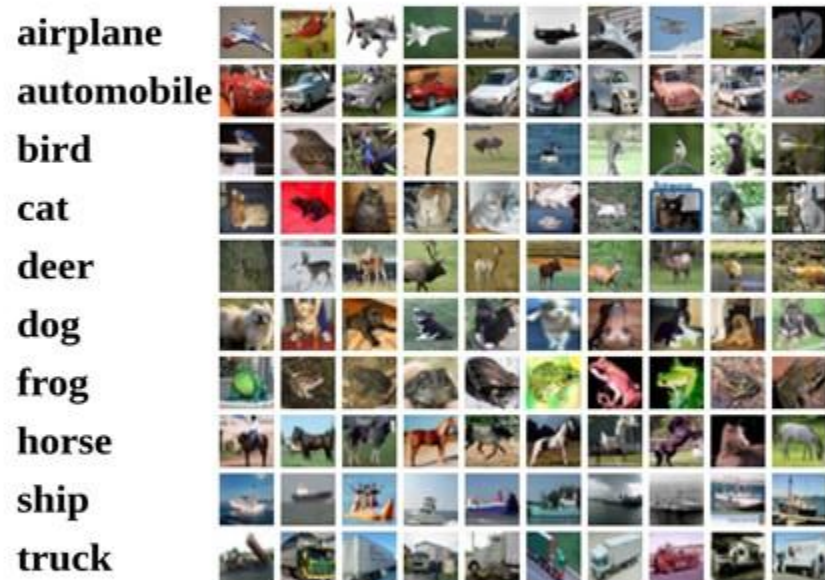


$$f(x, W) = Wx + b$$

Let we get a trained
weight Matrix

W
nx10

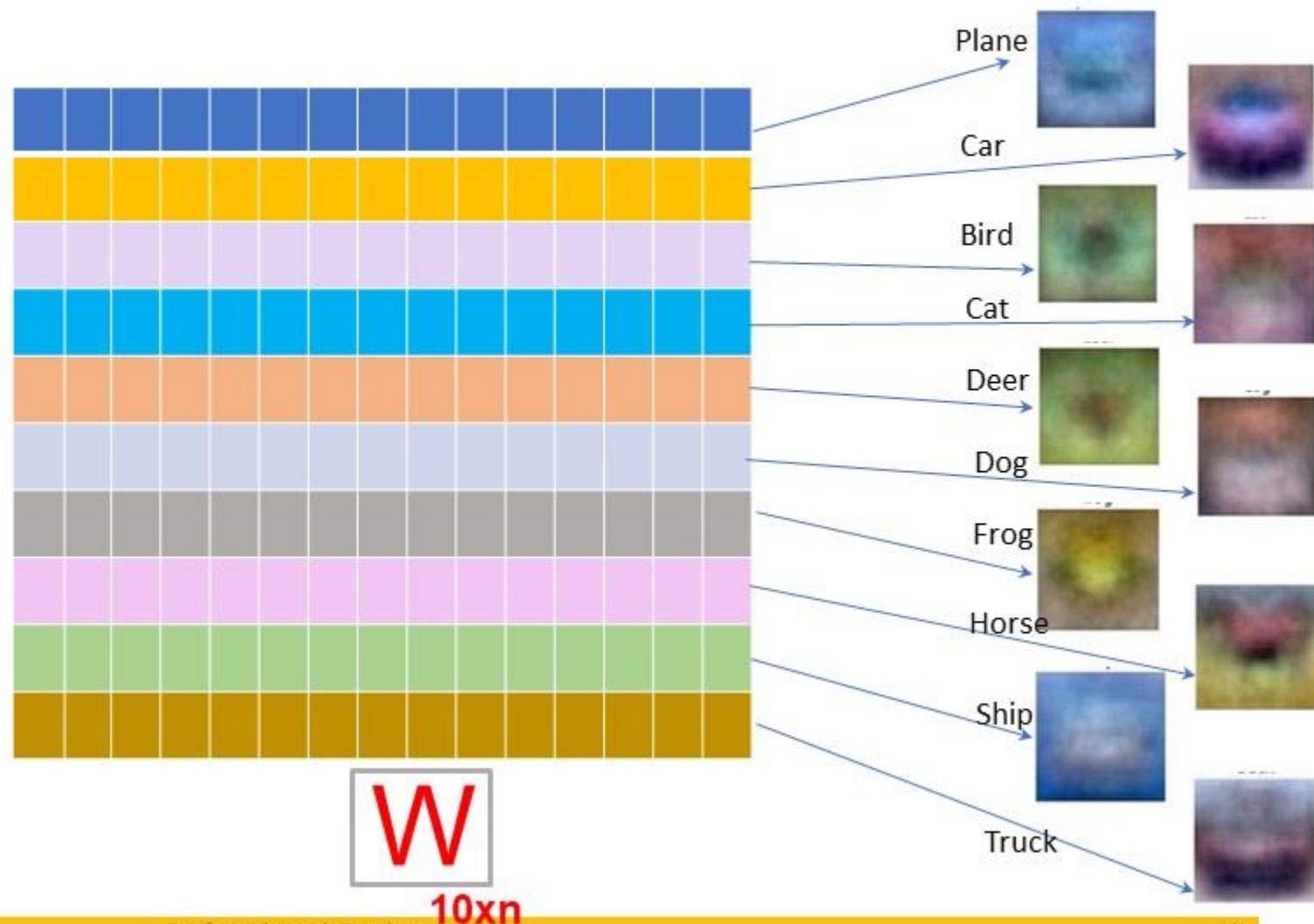
Interpretation of Linear Classifier



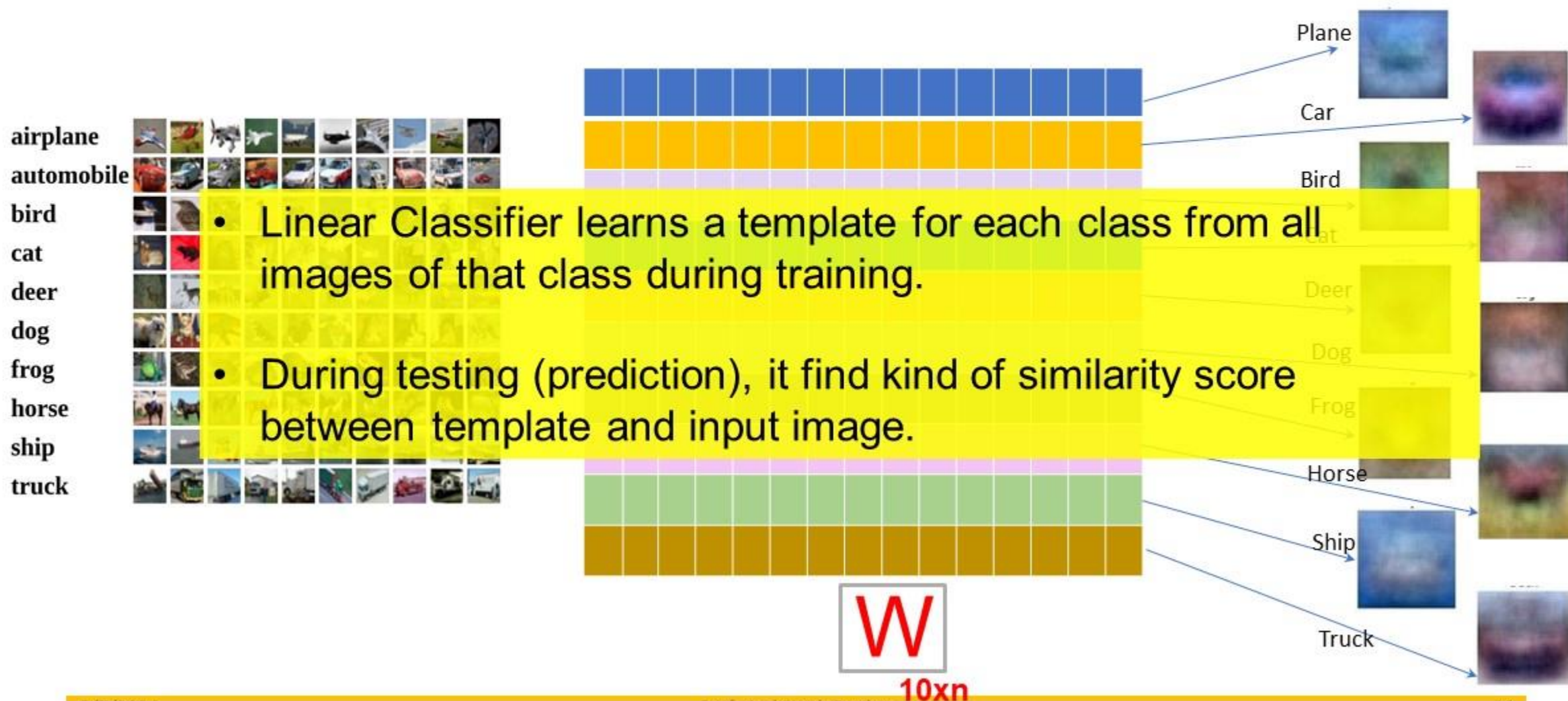
W
 $10 \times n$

- Convert each of the 10 rows of Matrix W in a matrix and display as an image.
- It is inverse operation of converting input image to column vector x

Interpretation of Linear Classifier



Interpretation of Linear Classifier



Interpretation of Linear Classifier

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

56
231
24
2

1.1
3.2
-1.2

-96.8
437.9
61.95

Cat score
Dog score
Ship score

W X W_0 $f(x, W) = Wx$

3x4 4x1 3x1 3x1

Size of weight matrix:
#classes x #rows in the input
i.e. 3x4

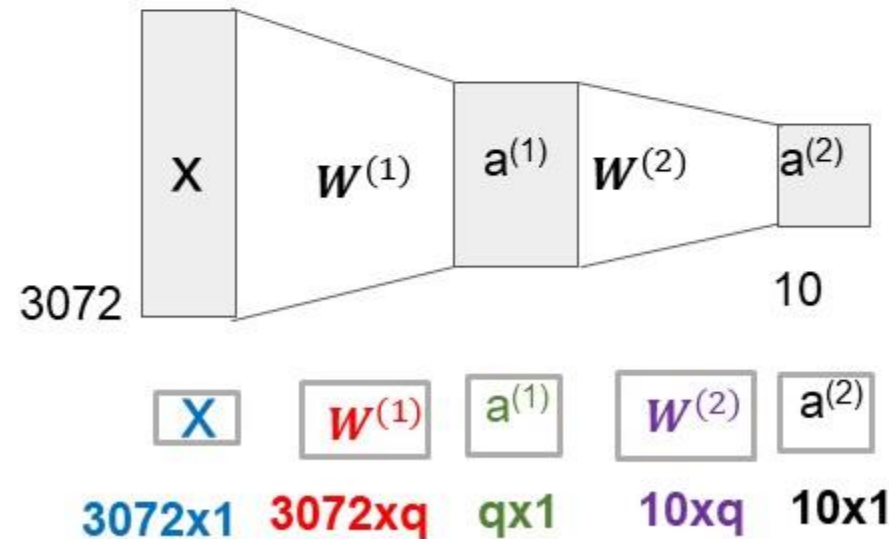
Multilayer Neural Networks

(Earlier) Linear function:

$$f = Wx$$

(Now) 2-Layer Neural
Network:

$$f = W^{(2)} \max(0, W^{(1)}x)$$



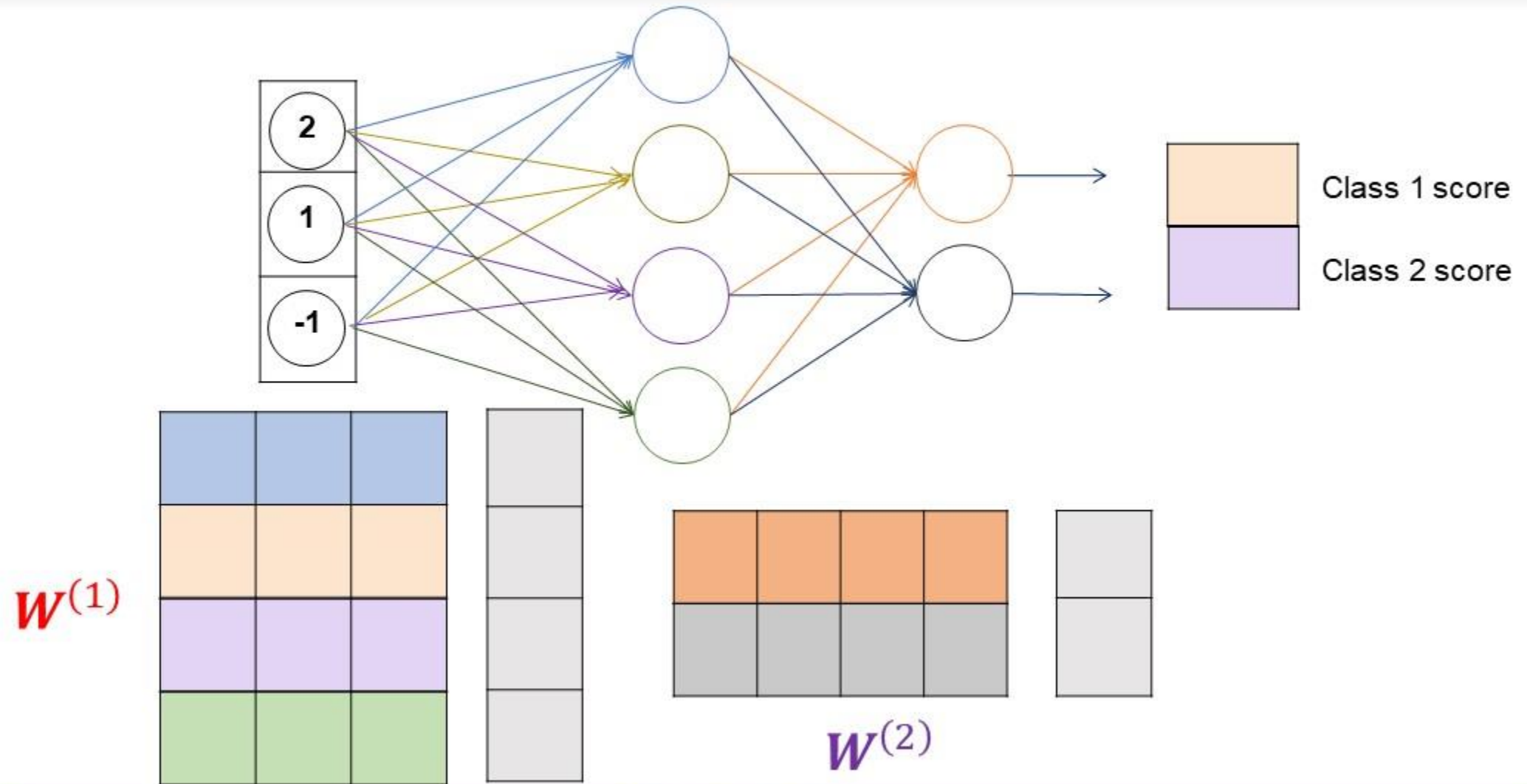
$$z^{(1)} = w^{(1)}x$$
$$a^{(1)} = \max(0, z^{(1)}) \quad \text{Non-Linear}$$

$$z^{(2)} = w^{(2)}a^{(1)}$$

$$f = a^{(2)} = z^{(2)} \quad \text{Linear}$$

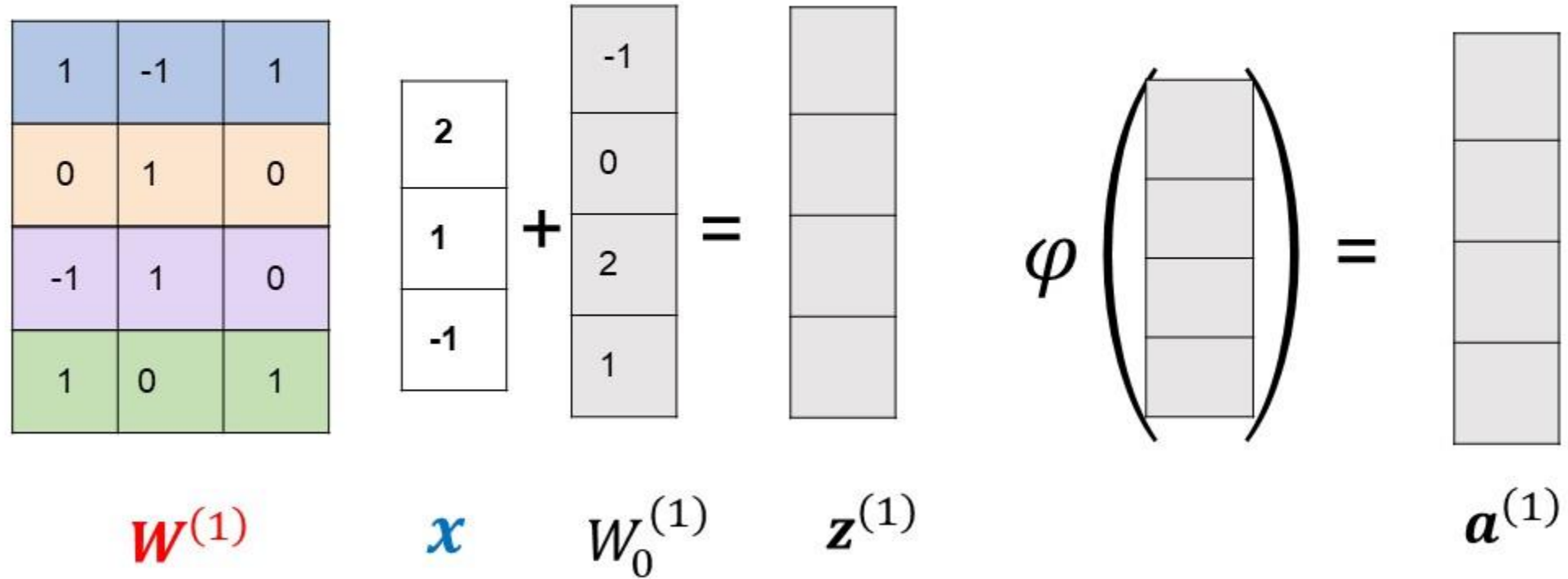
$$f = W^{(2)} \max(0, W^{(1)}x)$$

Multilayer Neural Networks



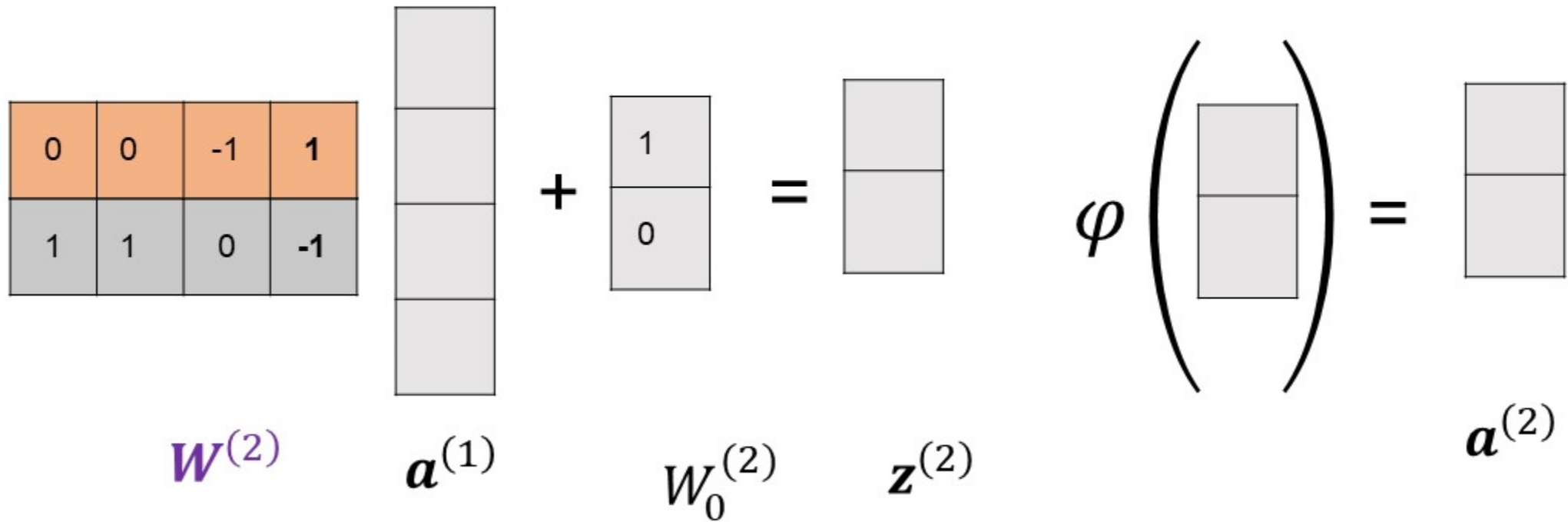
Multilayer Neural Networks

Layer 1:



Multilayer Neural Networks

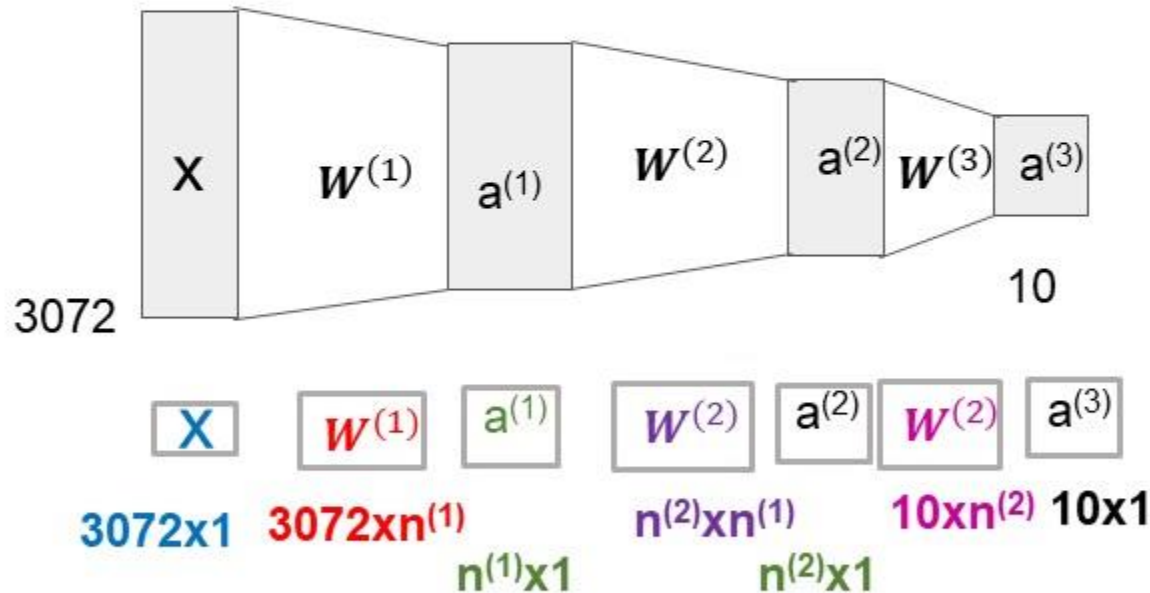
Layer 2:



Multilayer Neural Networks

3-Layer Neural Network:

$$f = \mathbf{W}^{(3)} \max(0, \mathbf{W}^{(2)} \max(0, \mathbf{W}^{(1)} \mathbf{x}))$$



$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)} \mathbf{x}$$

$$\mathbf{a}^{(1)} = \max(0, \mathbf{z}^{(1)}) \quad \text{ReLU}$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)} \mathbf{a}^{(1)}$$

$$\mathbf{a}^{(2)} = \max(0, \mathbf{z}^{(2)}) \quad \text{ReLU}$$

$$\mathbf{z}^{(3)} = \mathbf{W}^{(3)} \mathbf{a}^{(2)}$$

$$f = \mathbf{a}^{(3)} = \mathbf{z}^{(3)} \quad \text{Linear}$$

$$f = \mathbf{W}^{(3)} \max(0, \mathbf{W}^{(2)} \max(0, \mathbf{W}^{(1)} \mathbf{x}))$$

MLP: Multiple Data Points

Layer 1:

1	-1	1
0	1	0
-1	1	0
1	0	1

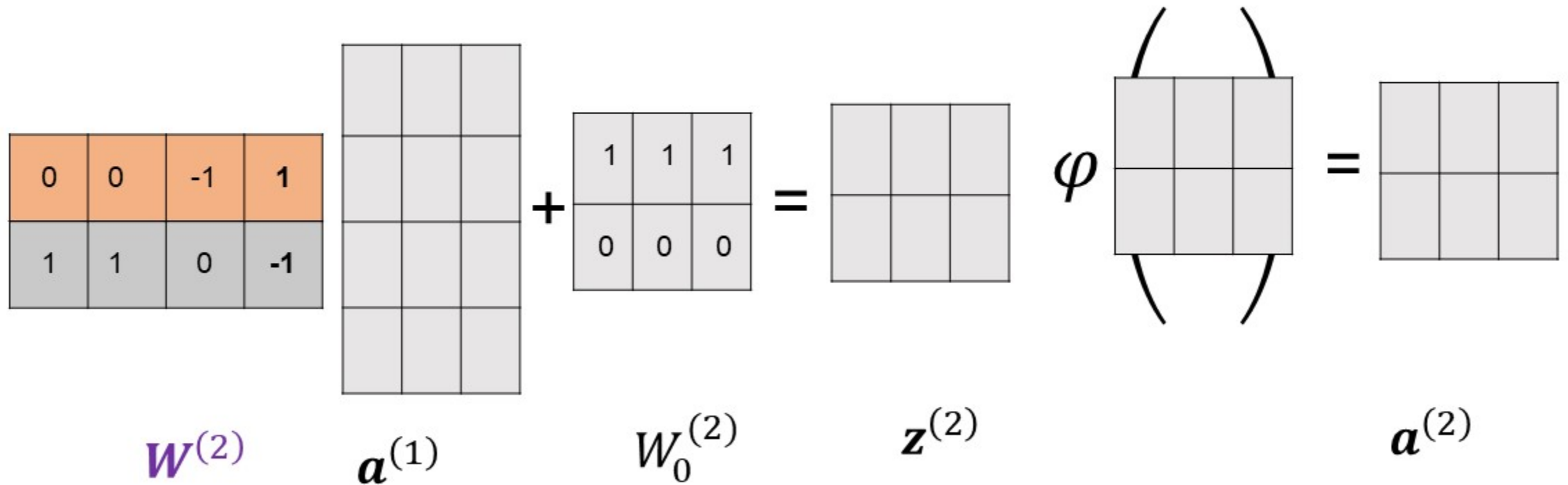
2	1	1
1	0	1
-1	1	0

-1	-1	-1
0	0	0
2	2	2
1	1	1

$W^{(1)} \quad x \quad W_0^{(1)} \quad = \quad z^{(1)} \quad \varphi \quad = \quad a^{(1)}$

Multilayer Neural Networks

Layer 2:





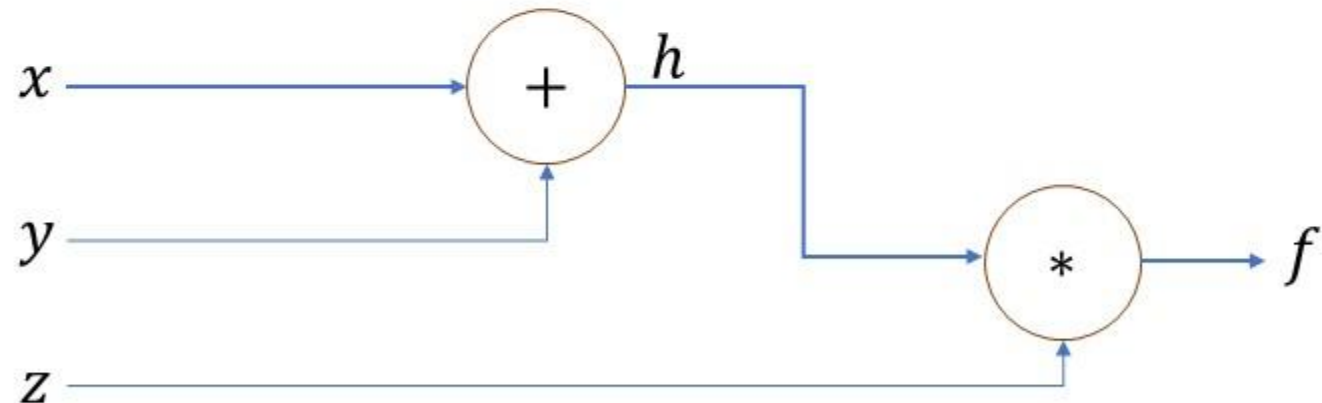
Computational Graph

➤ Example 1: $f(x, y, z) = (x + y)z$

Let $x = 3, y = -4, z = 5$

$$h = x + y,$$

$$f = hz$$



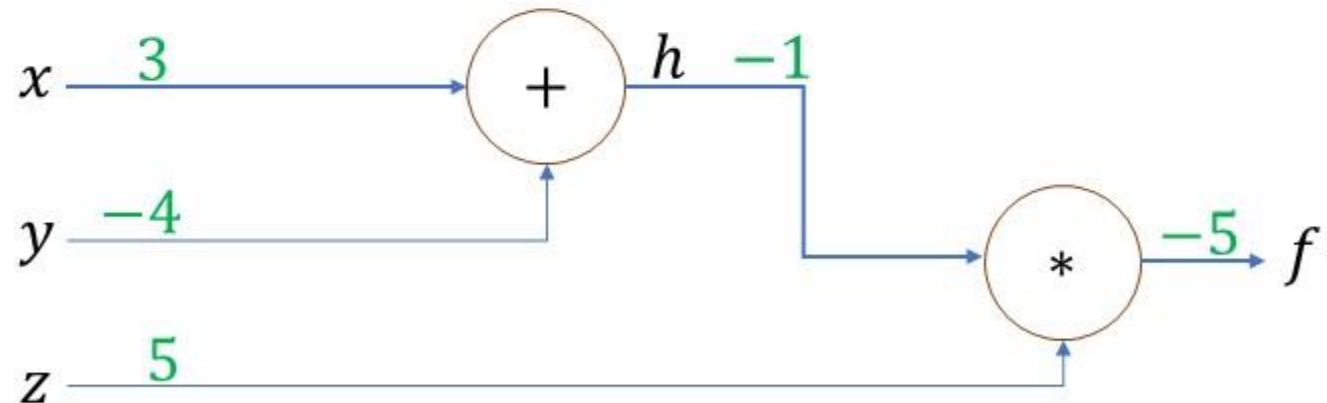
Computational Graph : Forward pass

➤ Example 1: $f(x, y, z) = (x + y)z$

Let $x = 3, y = -4, z = 5$

$h = x + y,$

$f = hz$

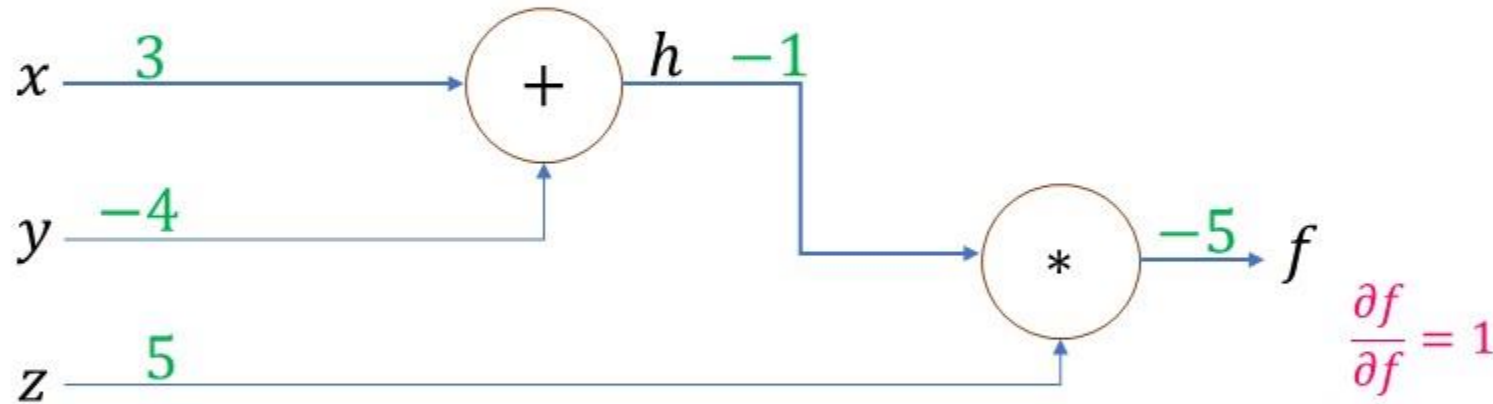


Computational Graph : Backward pass

➤ To find: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$$h = x + y,$$

$$f = hz$$

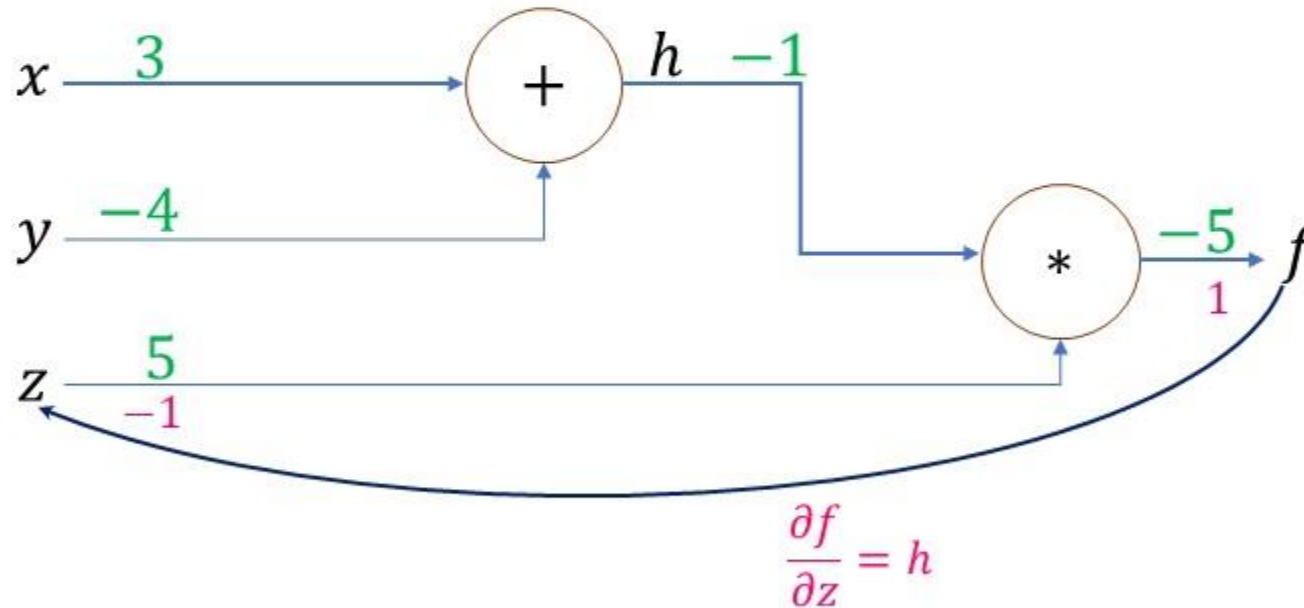


Computational Graph : Backward pass

➤ To find: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$$h = x + y,$$

$$f = hz$$

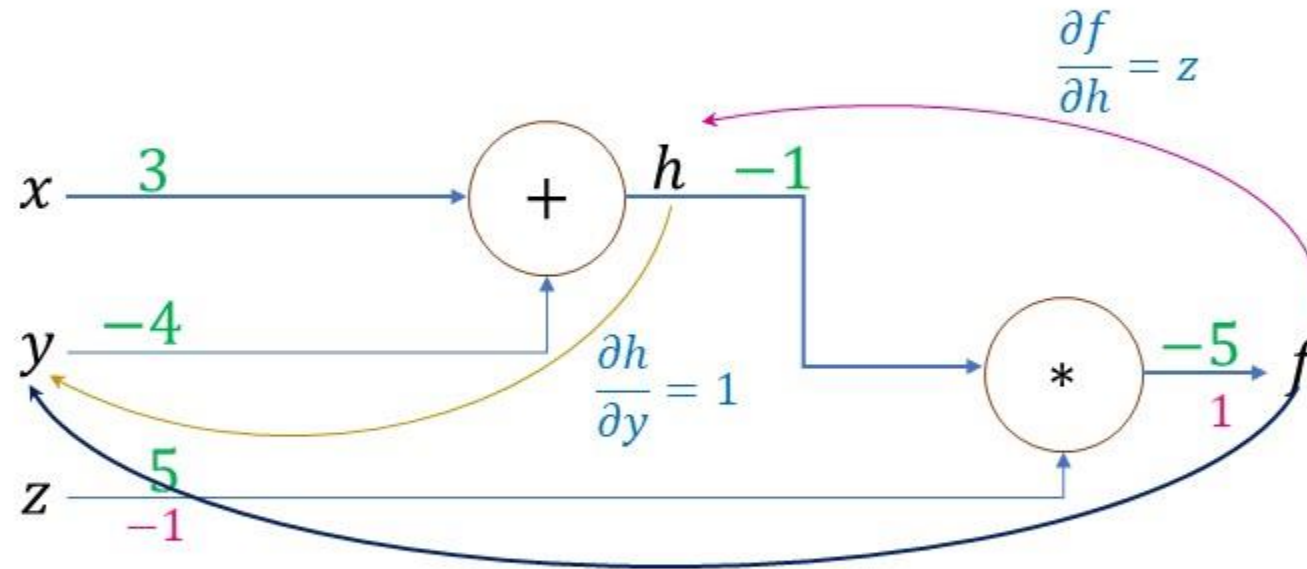


Computational Graph : Backward pass

➤ To find: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$$h = x + y,$$

$$f = hz$$



➤ Chain Rule: $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial h} \times \frac{\partial h}{\partial y}$ $\frac{\partial f}{\partial y} = ?$

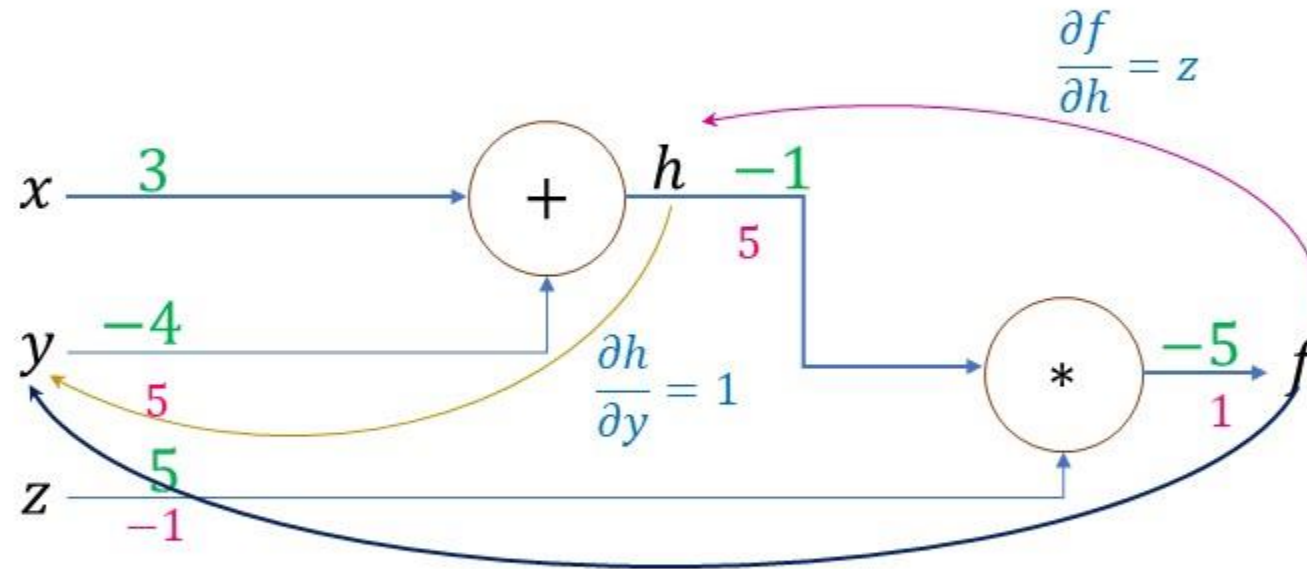
$$\frac{\partial f}{\partial y} = z \times 1 = 5$$

Computational Graph : Backward pass

➤ To find: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

$$h = x + y,$$

$$f = hz$$

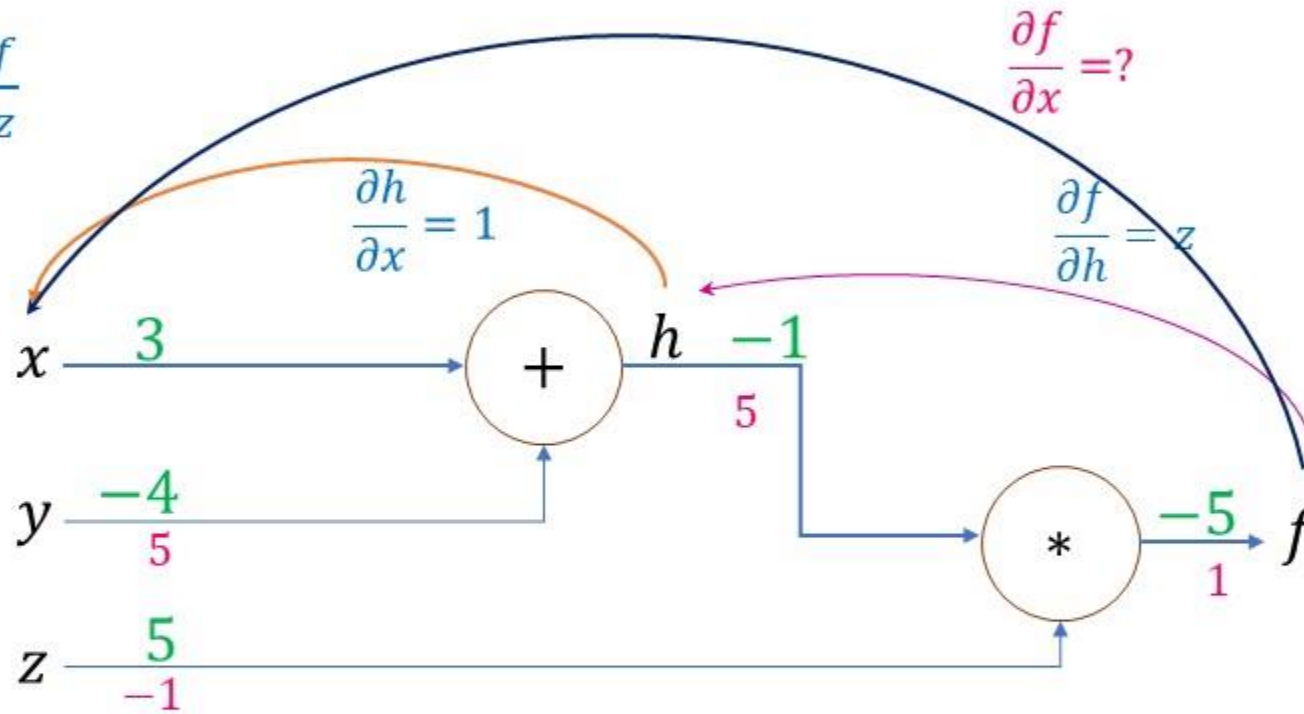


➤ Chain Rule: $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial h} \times \frac{\partial h}{\partial y}$

$$\frac{\partial f}{\partial y} = z \times 1 = 5$$

Computational Graph: Backward pass

➤ To find: $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$



$$h = x + y,$$

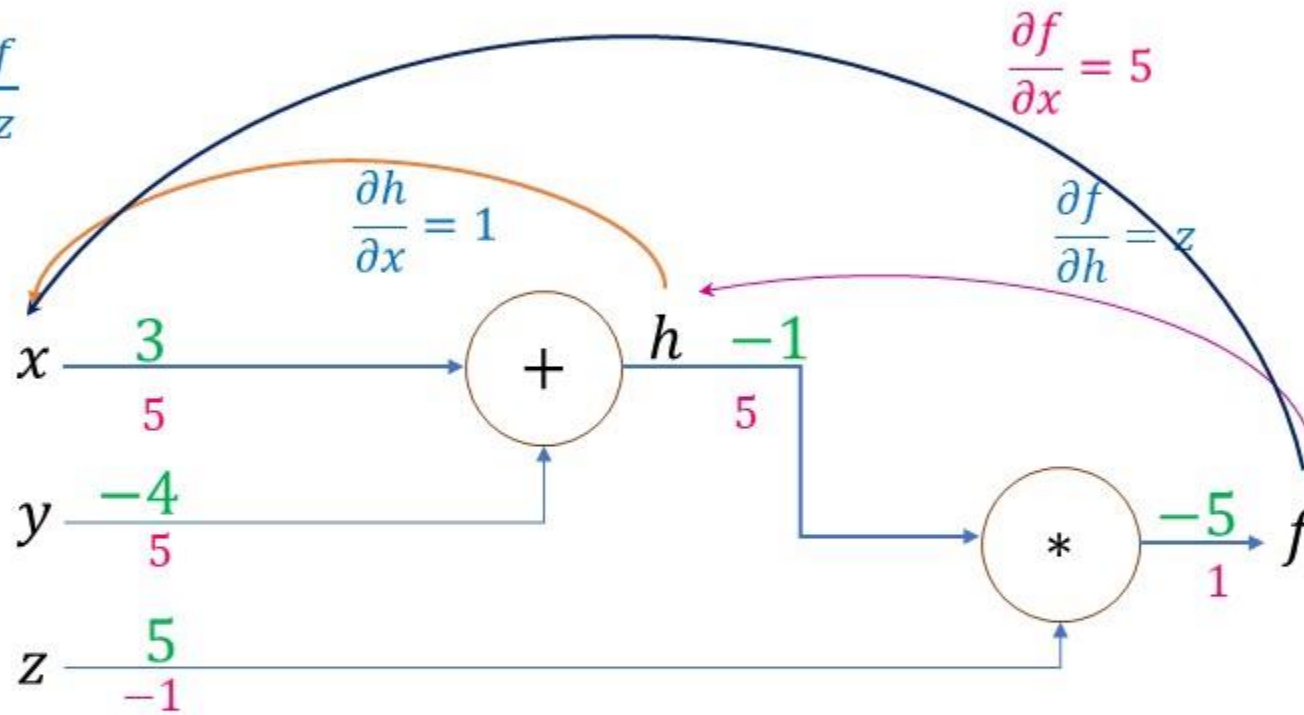
$$f = hz$$

➤ Chain Rule: $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial h} \times \frac{\partial h}{\partial x}$

$$\frac{\partial f}{\partial x} = z \times 1 = 5$$

Computational Graph: Backward pass

➤ To find: $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$



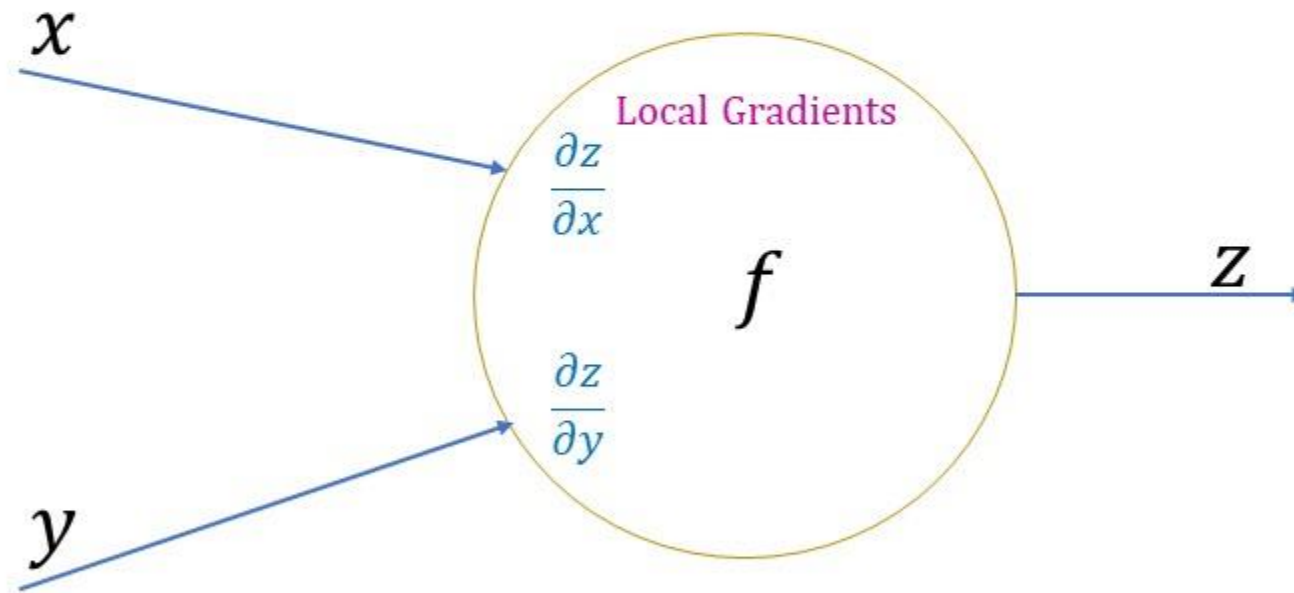
$$h = x + y,$$

$$f = hz$$

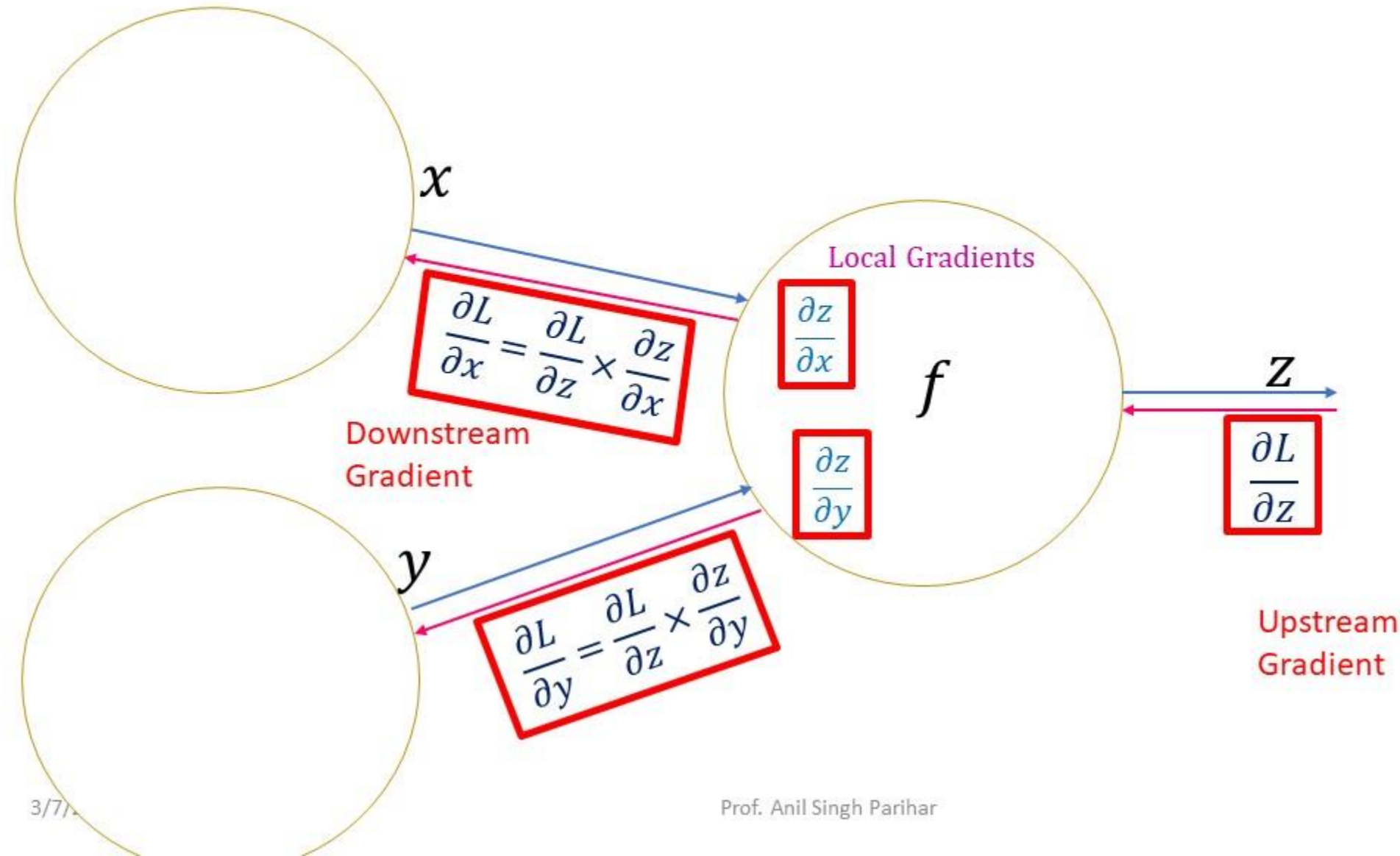
➤ Chain Rule: $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial h} \times \frac{\partial h}{\partial x}$

$$\frac{\partial f}{\partial x} = z \times 1 = 5$$

Computational Graph: Local Gradients

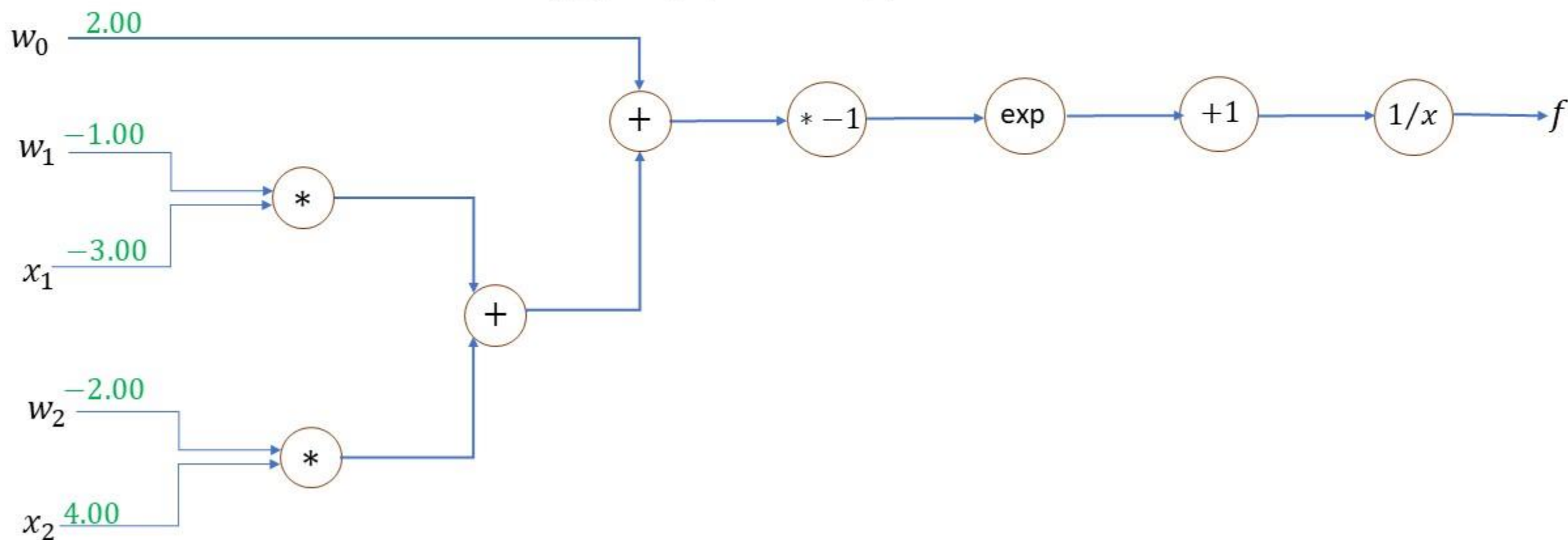


Backpropagation: Gradient flow



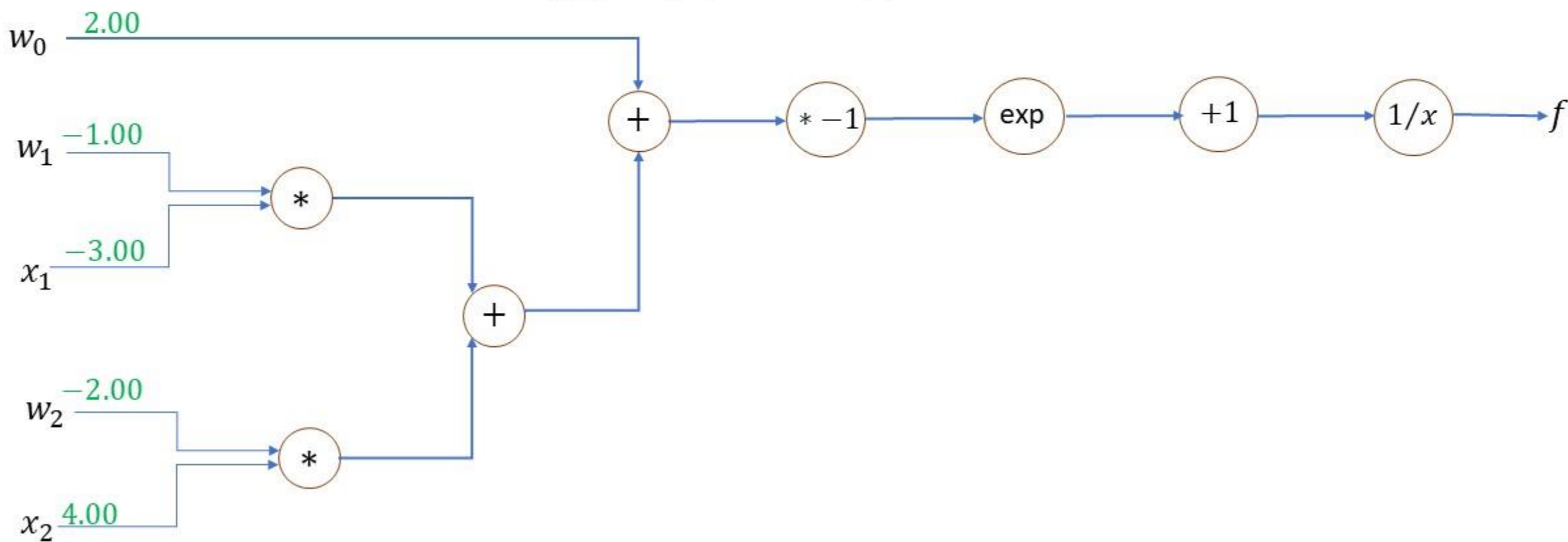
Computational Graph: Forward Pass

Example 2: $f(x, y, z) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2x_2)}}$



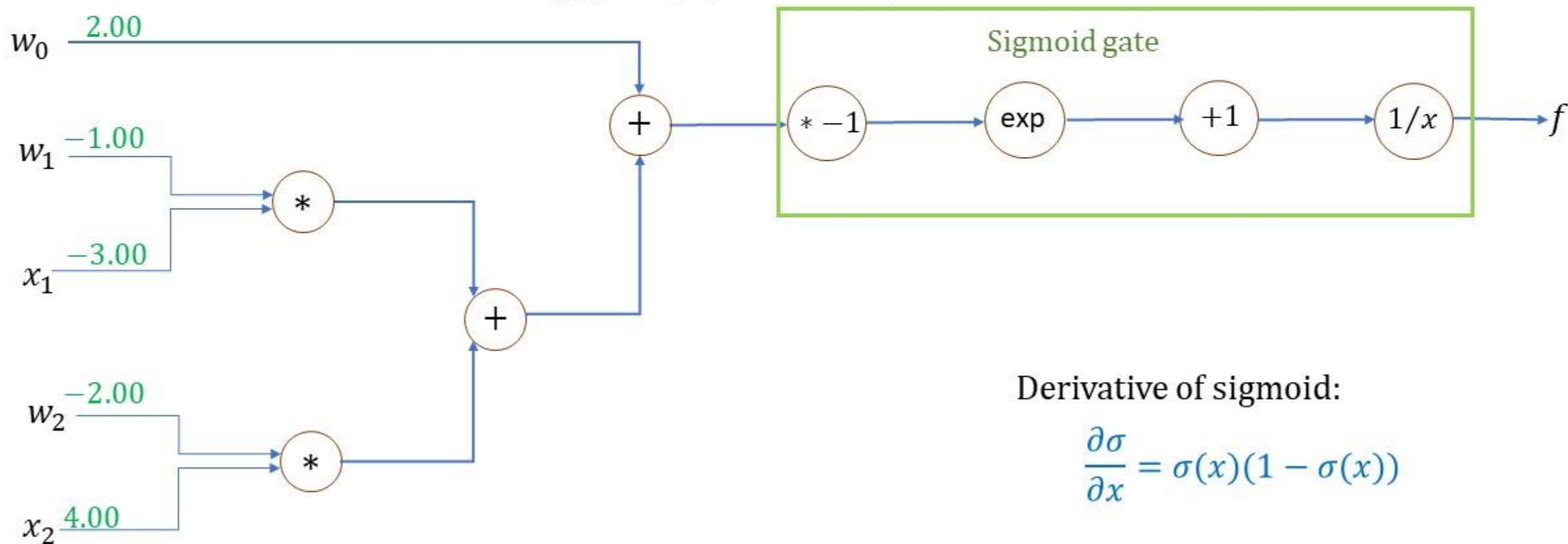
Computational Graph: Backward Pass

Example 2: $f(x, y, z) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}}$



Computational Graph: Backward Pass

Example 2: $f(x, y, z) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2x_2)}}$

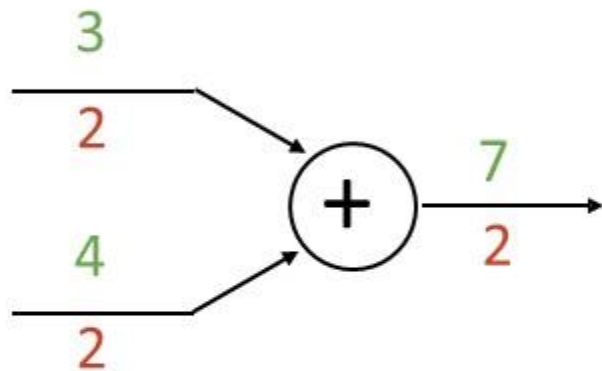


Derivative of sigmoid:

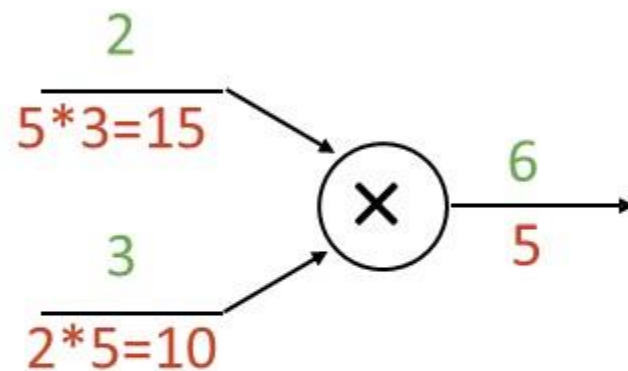
$$\frac{\partial \sigma}{\partial x} = \sigma(x)(1 - \sigma(x))$$

Patterns in Gradient Flow

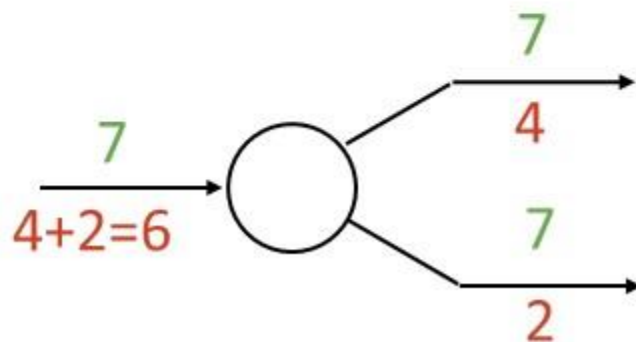
add gate: gradient distributor



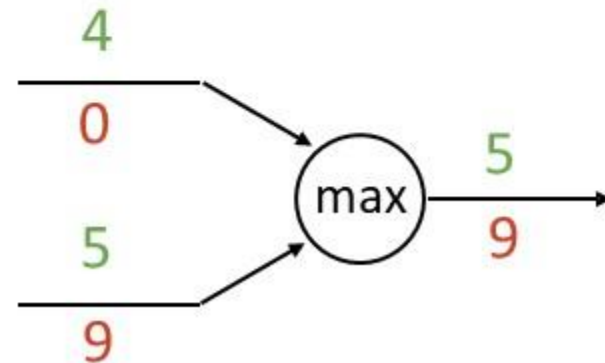
mul gate: “swap multiplier”



copy gate: gradient adder



max gate: gradient router



Backprop Implementation

Forward pass:
Compute output

```
def f(w0, x0, w1, x1, w2):
```

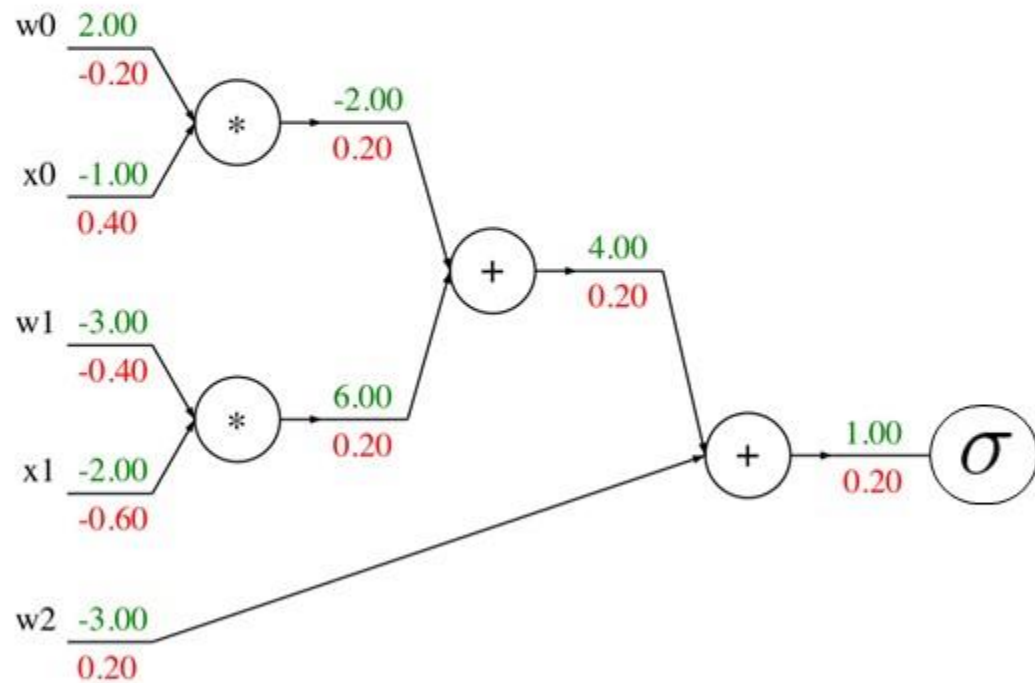
```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

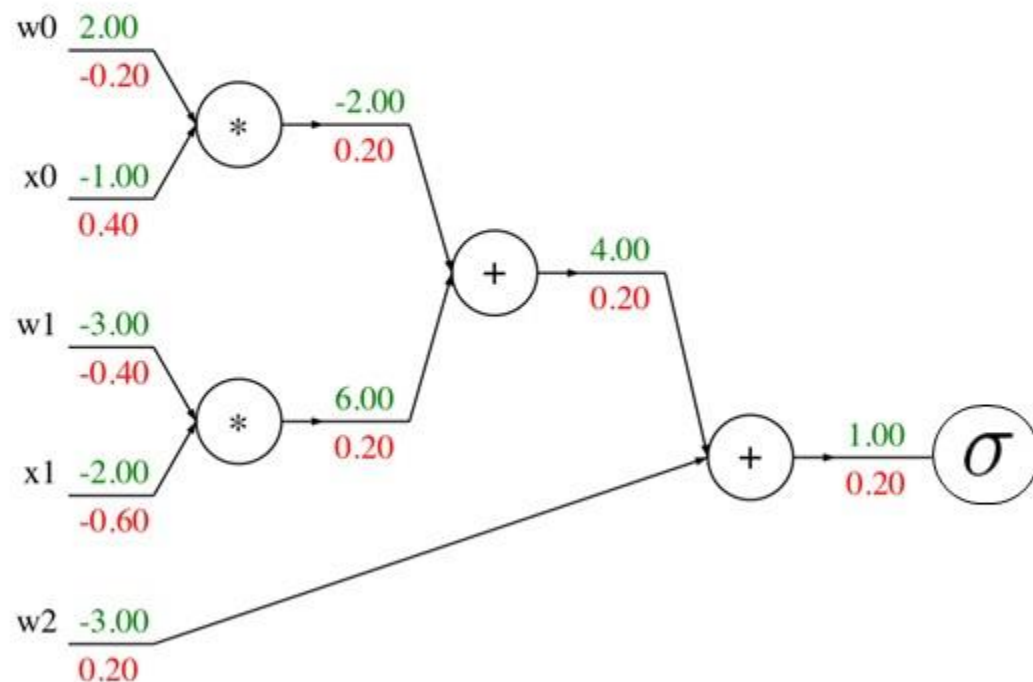
```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```



Backprop Implementation

Forward pass:
Compute output



Backward pass:
Compute grads

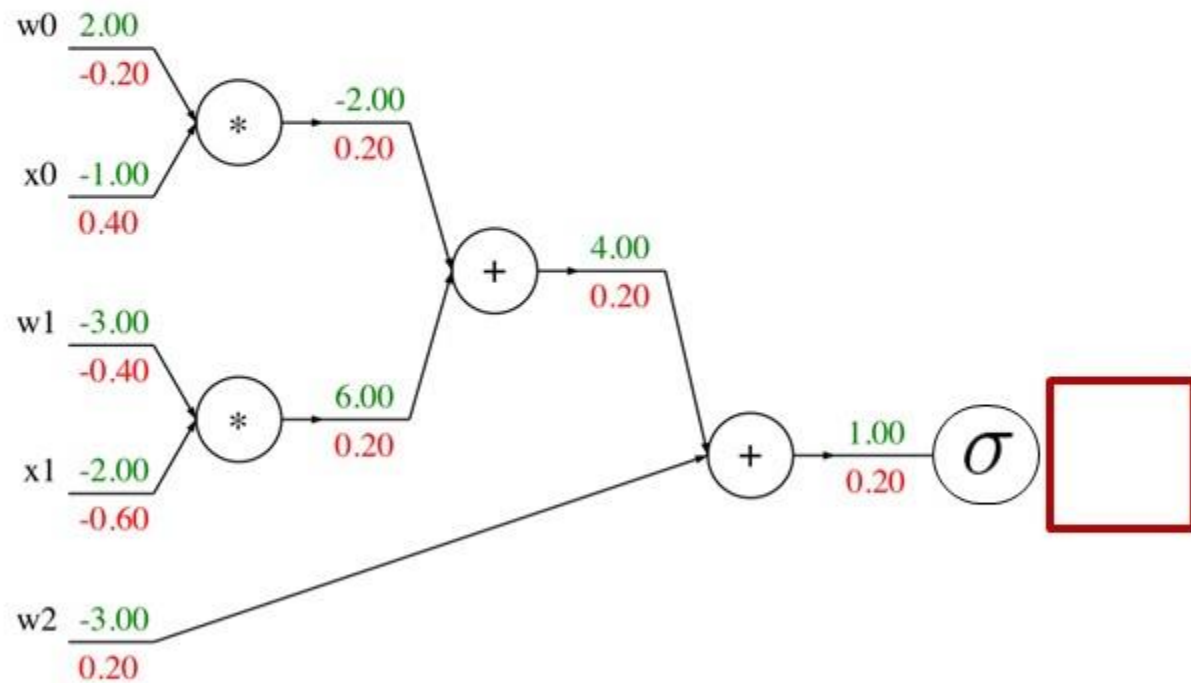
```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
    s1 = w1 * x1
    s2 = s0 + s1
    s3 = s2 + w2
    L = sigmoid(s3)
```

```
    grad_L = 1.0
    grad_s3 = grad_L * (1 - L) * L
    grad_w2 = grad_s3
    grad_s2 = grad_s3
    grad_s1 = grad_s2
    grad_w1 = grad_s1 * x1
    grad_x1 = grad_s1 * w1
    grad_w0 = grad_s0 * x0
    grad_x0 = grad_s0 * w0
```

Backprop Implementation

Forward pass:
Compute output



Base case

```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

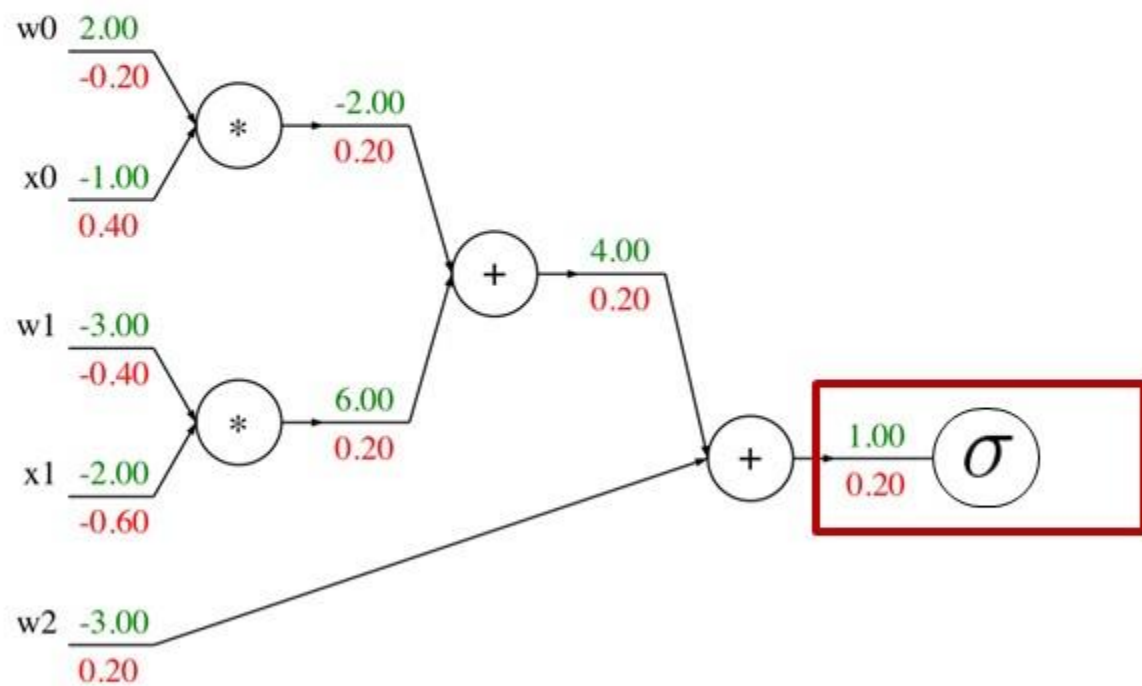
```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```


Backprop Implementation

Forward pass:
Compute output



Sigmoid

```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

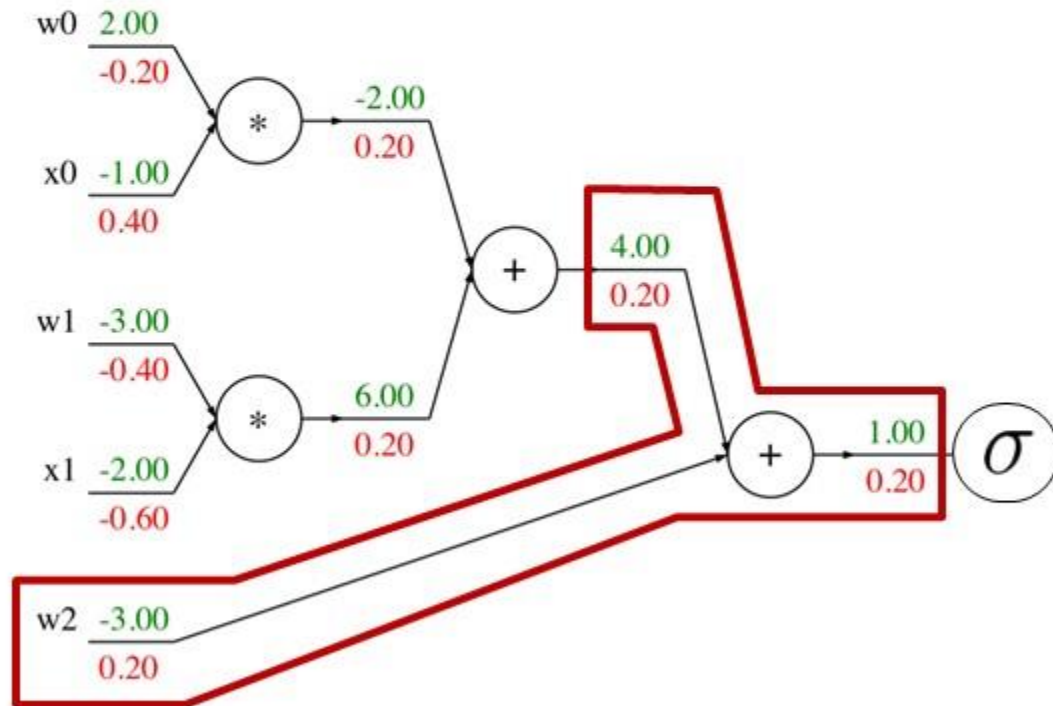
```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Backprop Implementation

Forward pass:
Compute output



```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

```
    grad_x1 = grad_s1 * w1
```

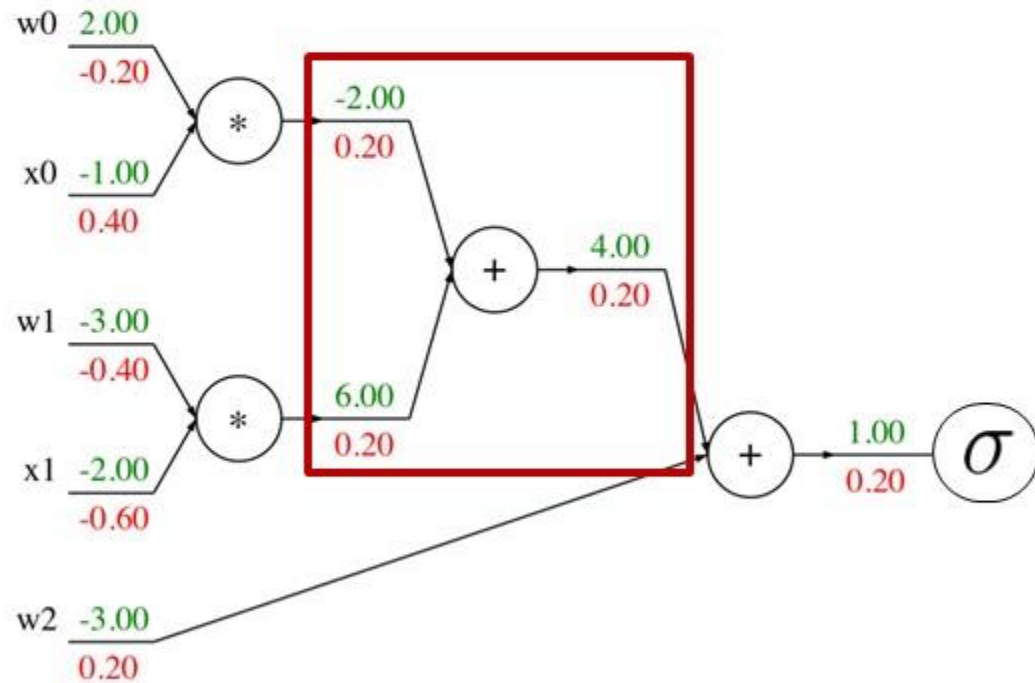
```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Add

Backprop Implementation

Forward pass:
Compute output



```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

```
    grad_x1 = grad_s1 * w1
```

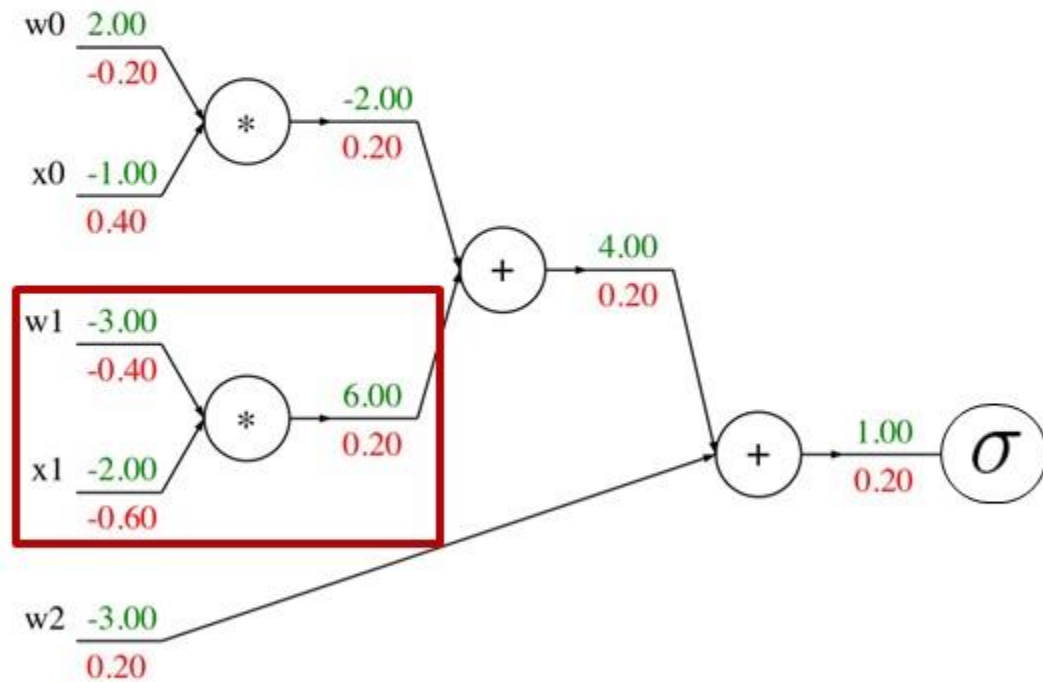
```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Add

Backprop Implementation

Forward pass:
Compute output



```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

```
    grad_x1 = grad_s1 * w1
```

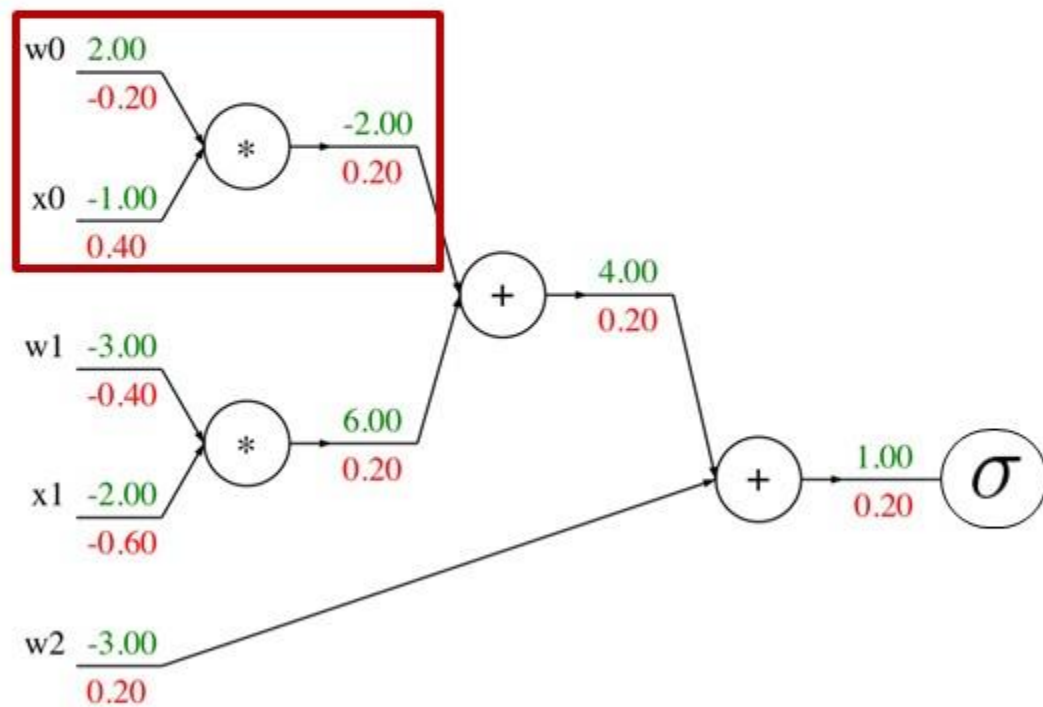
```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Multiply

Backprop Implementation

Forward pass:
Compute output



```
def f(w0, x0, w1, x1, w2):
```

```
    s0 = w0 * x0
```

```
    s1 = w1 * x1
```

```
    s2 = s0 + s1
```

```
    s3 = s2 + w2
```

```
    L = sigmoid(s3)
```

```
    grad_L = 1.0
```

```
    grad_s3 = grad_L * (1 - L) * L
```

```
    grad_w2 = grad_s3
```

```
    grad_s2 = grad_s3
```

```
    grad_s0 = grad_s2
```

```
    grad_s1 = grad_s2
```

```
    grad_w1 = grad_s1 * x1
```

```
    grad_x1 = grad_s1 * w1
```

```
    grad_w0 = grad_s0 * x0
```

```
    grad_x0 = grad_s0 * w0
```

Multiply

Vector Derivatives

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If x changes by a small amount, how much will y change?

$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

Derivative is **Gradient**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N,$$
$$\left(\frac{\partial y}{\partial x} \right)_i = \frac{\partial y}{\partial x_i}$$

For each element of x , if it changes by a small amount then how much will y change?

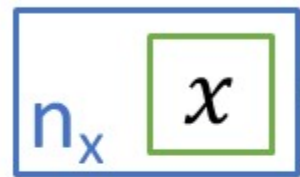
$$x \in \mathbb{R}^N, y \in \mathbb{R}^M$$

Derivative is **Jacobian**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^{N \times M},$$
$$\left(\frac{\partial y}{\partial x} \right)_{ij} = \frac{\partial y_j}{\partial x_i}$$

For each element of x , if it changes by a small amount then how much will each element of y change?

Backpropagation with Vectors

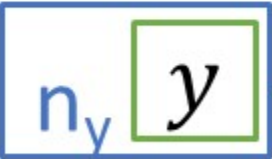
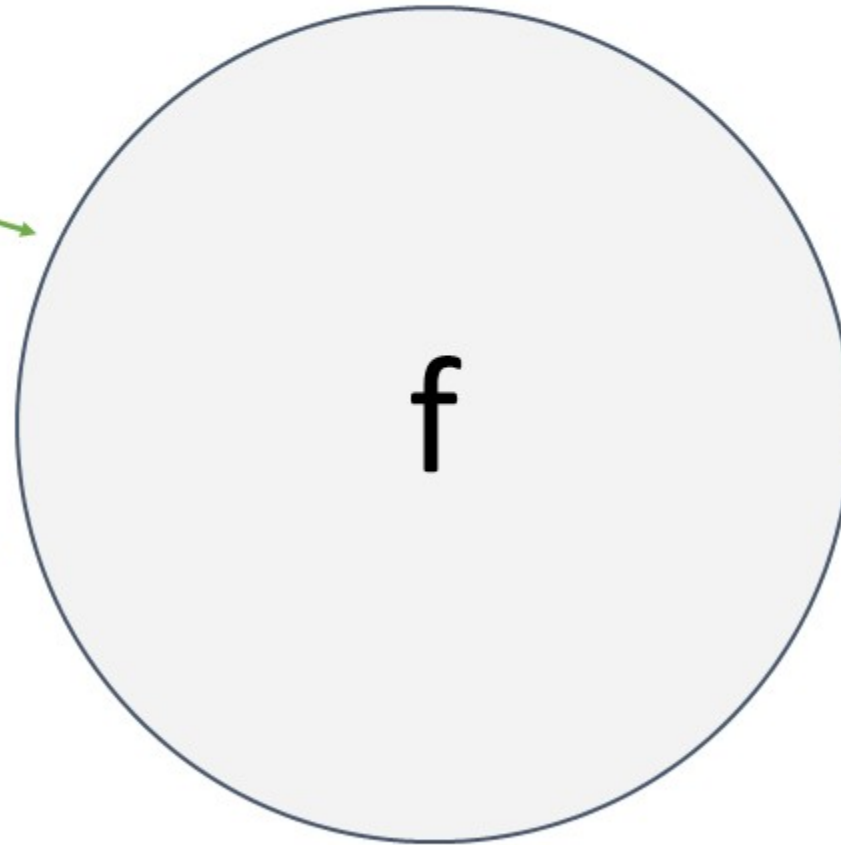


n_x is size of vector x

Loss L still a scalar!

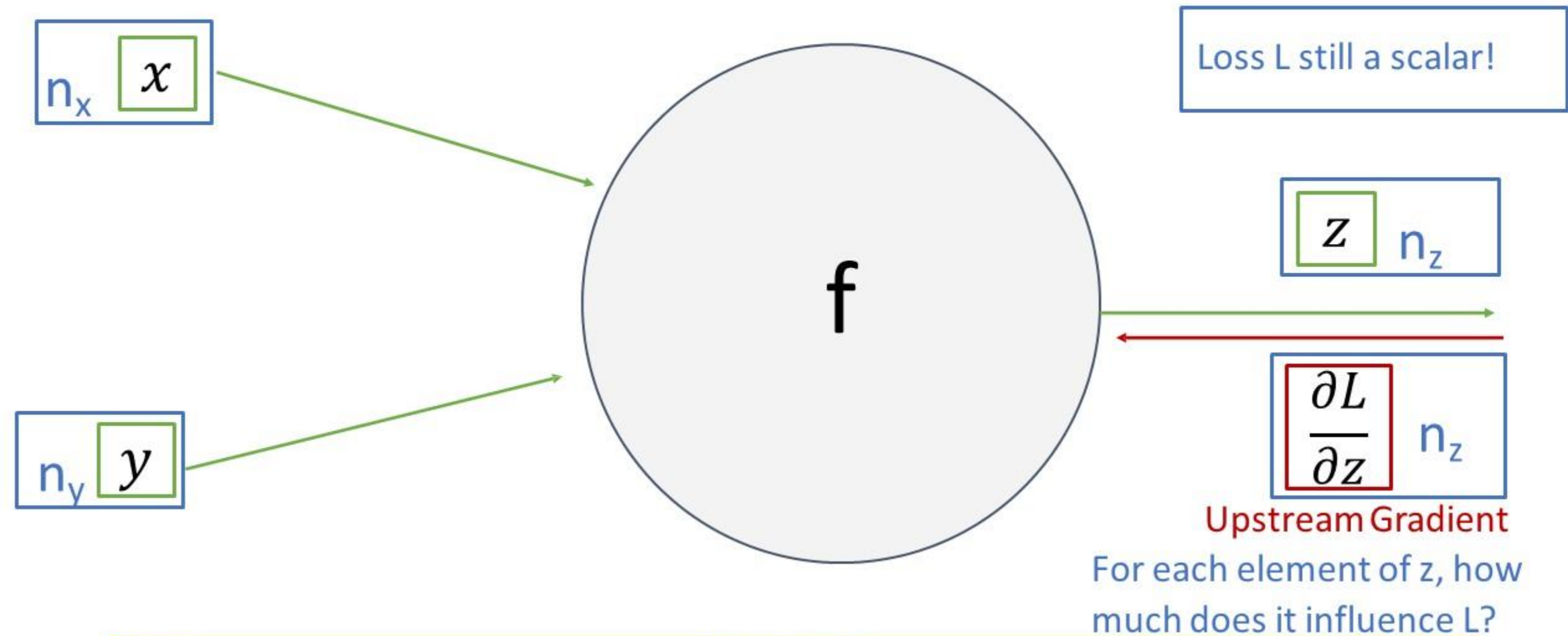


n_z is size of vector z

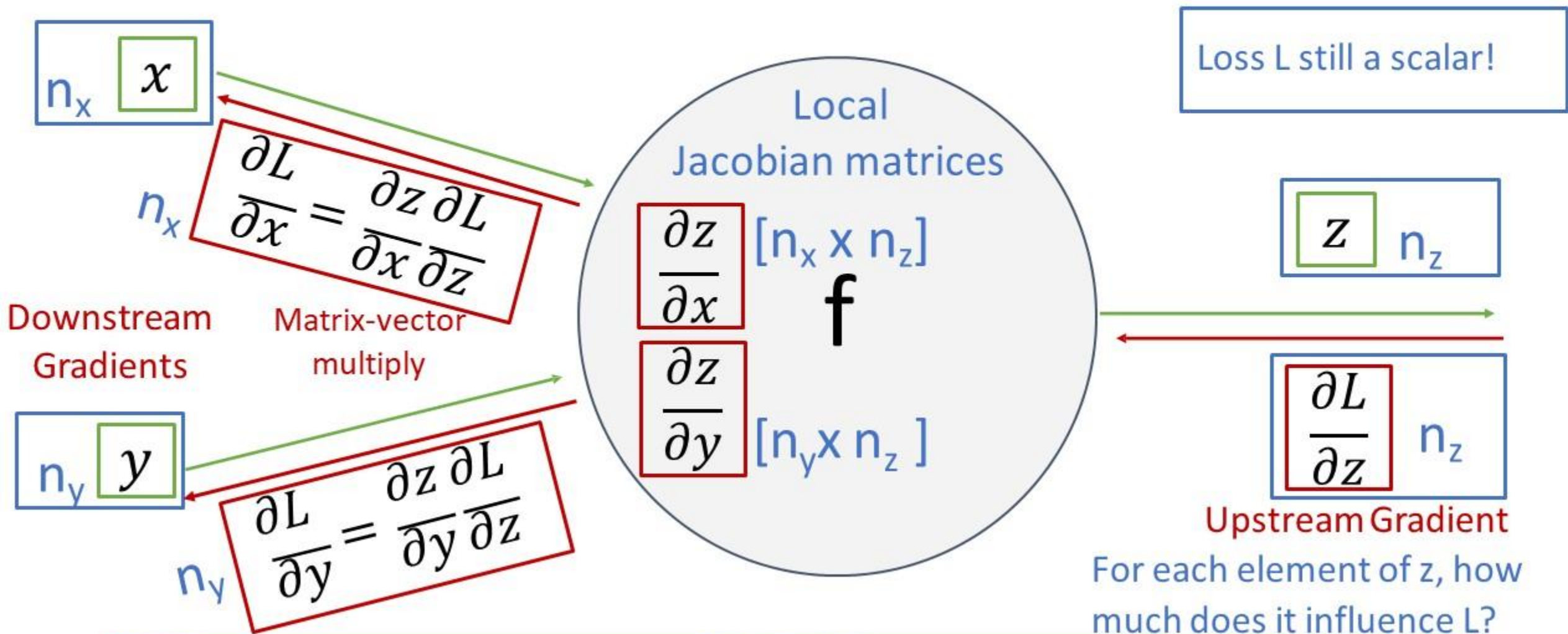


n_y is size of vector y

Backpropagation with Vectors



Backpropagation with Vectors



Backpropagation with Vectors

Jacobian is **sparse**: off-diagonal entries all zero! Never **explicitly** form Jacobian; instead use **implicit** multiplication

4D input x:

$\begin{bmatrix} 1 \\ -2 \\ 3 \\ -1 \end{bmatrix}$

$f(x) = \max(0, x)$
(*elementwise*)

4D output y:

$\begin{bmatrix} 1 \\ 0 \\ 3 \\ 0 \end{bmatrix}$

Local gradient

Jacobian dy/dx

4D dL/dy :

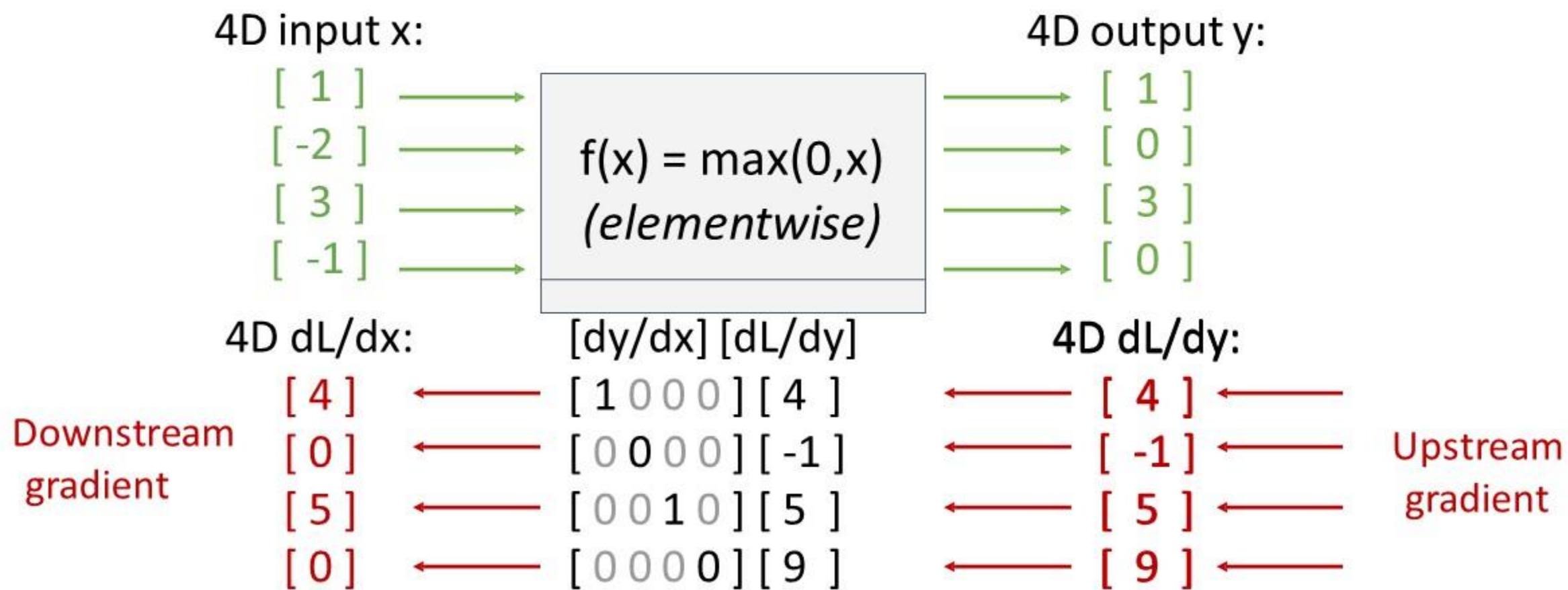
$\begin{bmatrix} 4 \\ -1 \\ 5 \\ 9 \end{bmatrix}$

Upstream
gradient

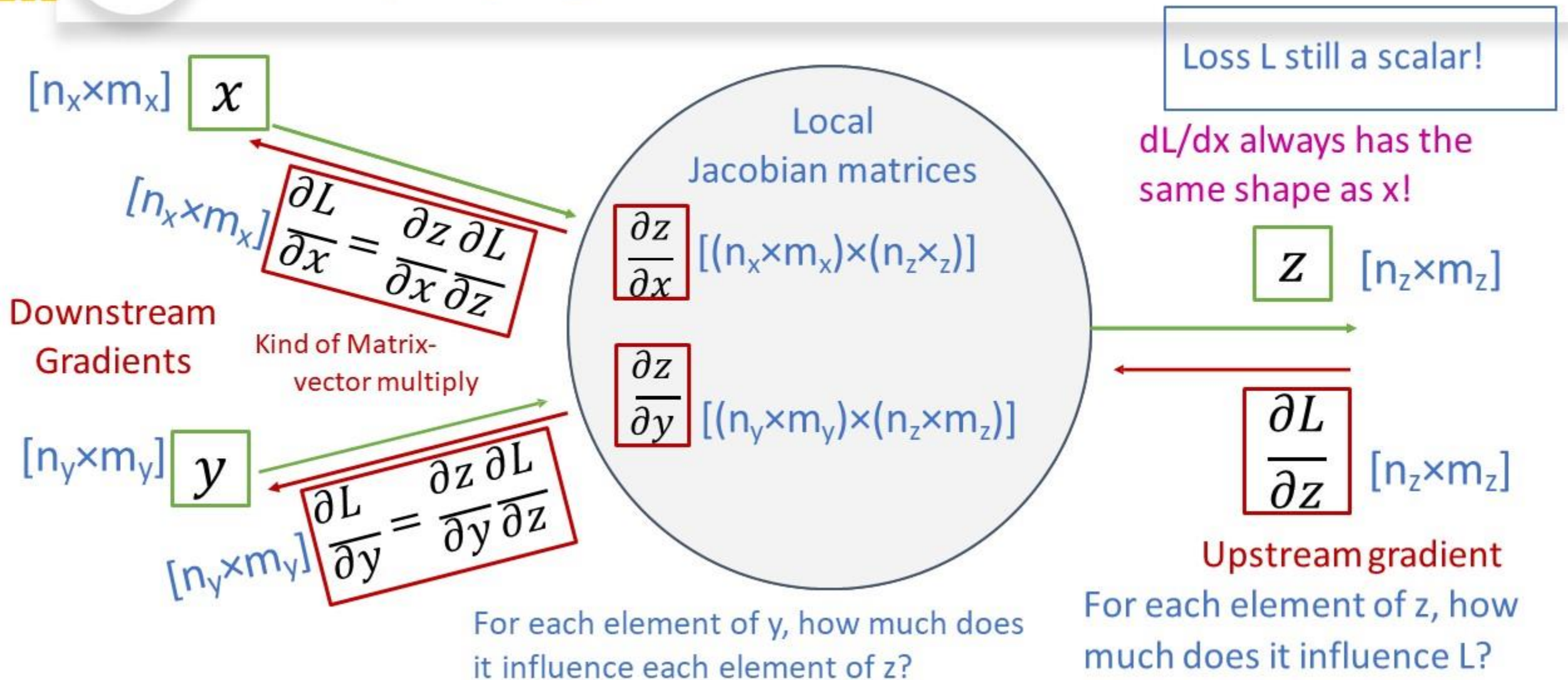
Size of local gradient is $[n_x \times n_y]$, i.e. 4×4 .
However, if we change x_i little bit then only y_i will change and others remain same.
Thus, all non diagonal element are 0.

$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$

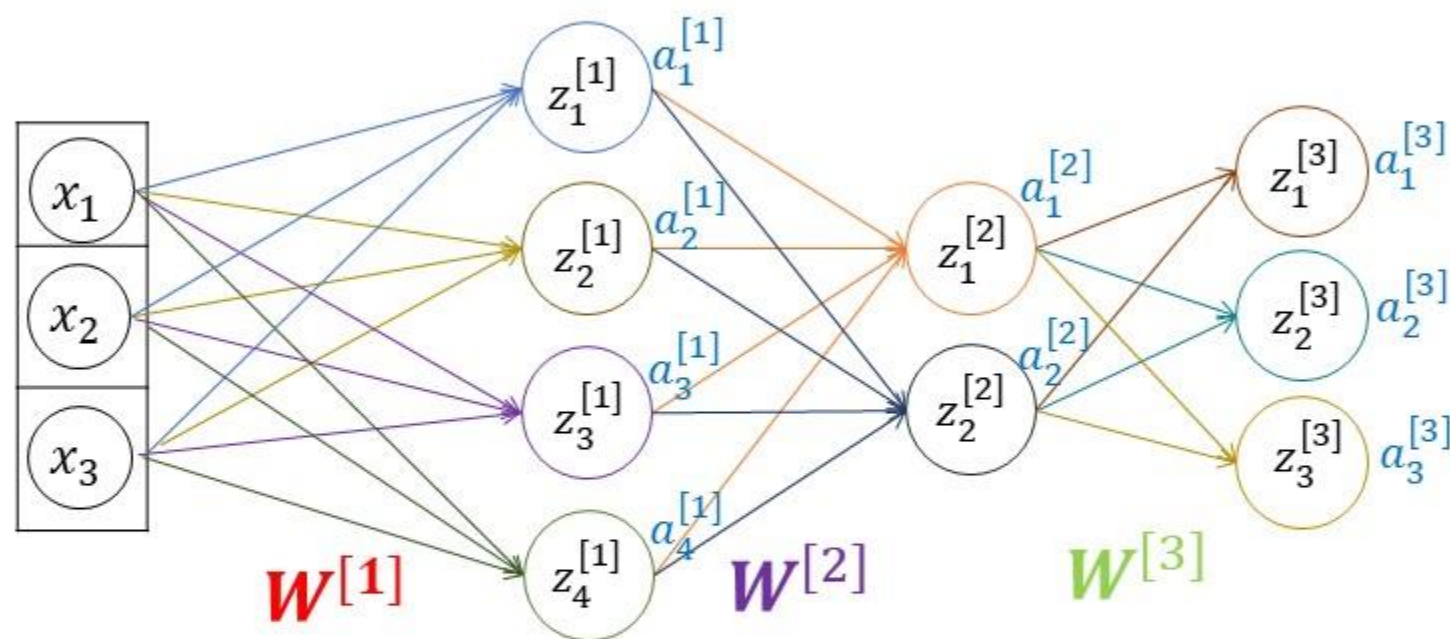
Backpropagation with Vectors



Backpropagation with Matrices or Tensors



ML-ANN: Forward



Target:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]} \mathbf{x} + \mathbf{w}_0^{[1]}$$

$$\mathbf{a}^{[1]} = g_1(\mathbf{z}^{[1]})$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]} \mathbf{a}^{[1]} + \mathbf{w}_0^{[2]}$$

$$\mathbf{a}^{[2]} = g_2(\mathbf{z}^{[2]})$$

$$\mathbf{z}^{[3]} = \mathbf{W}^{[3]} \mathbf{a}^{[2]} + \mathbf{w}_0^{[3]}$$

$$\mathbf{a}^{[3]} = g_2(\mathbf{z}^{[3]})$$

ML-ANN: Loss Calculation

Consider Cross-Entropy Loss:

$$\mathcal{L}(a, y) = -y \log(a) - (1 - y) \log(1 - a)$$

Activation function for last layer:

$$a = f(z) = \frac{1}{1 + \exp(-z)}$$

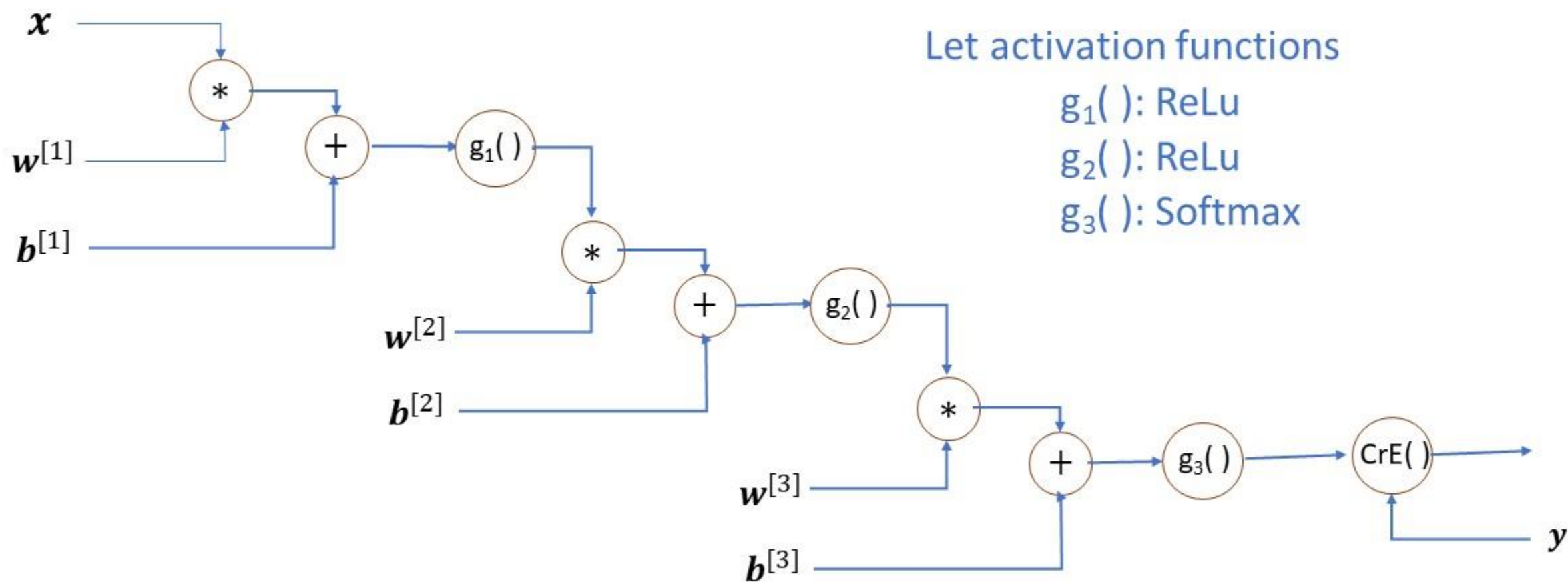
Derivatives:

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{-y}{a} + \frac{(1 - y)}{(1 - a)}$$

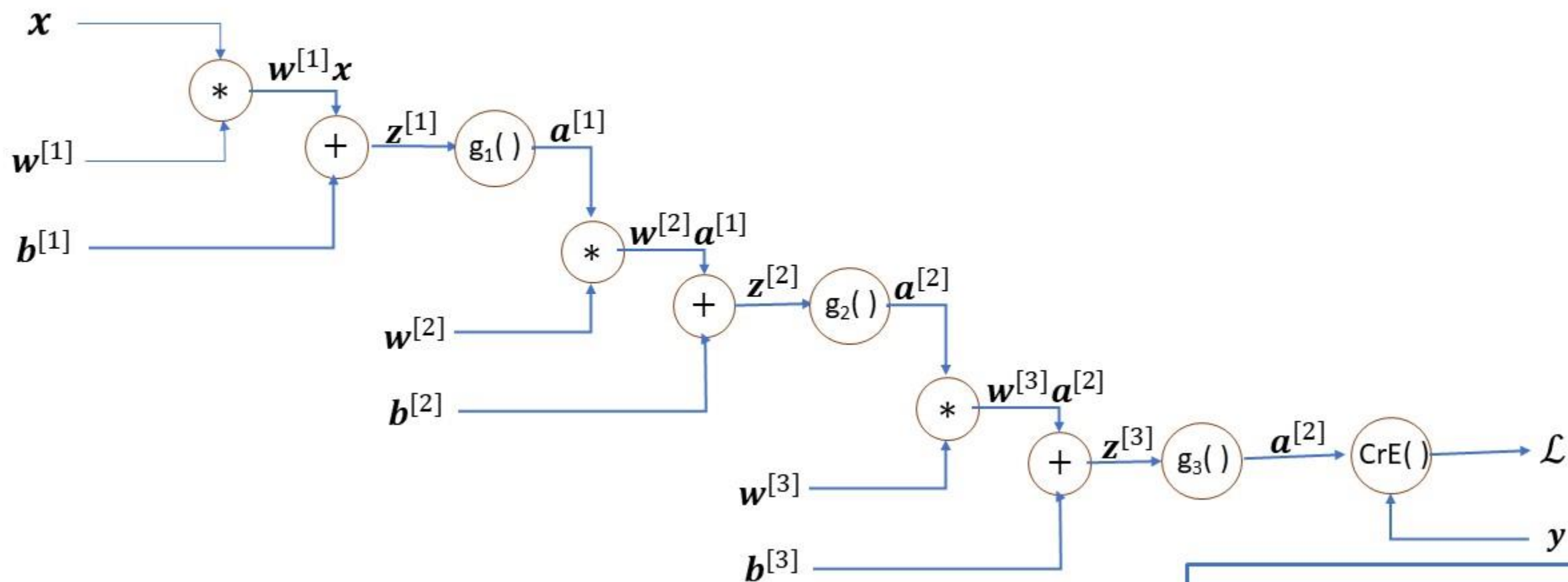
$$\frac{\partial a}{\partial z} = \left(\frac{1}{1 + \exp(-z)} \right) \left(\frac{\exp(-z)}{1 + \exp(-z)} \right) = \left(\frac{1}{1 + \exp(-z)} \right) \left(1 - \frac{1}{1 + \exp(-z)} \right) = a(1 - a)$$

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial z} = \left(\frac{-y}{a} + \frac{(1 - y)}{(1 - a)} \right) a(1 - a) = a - y$$

ML-ANN: Computational Graph

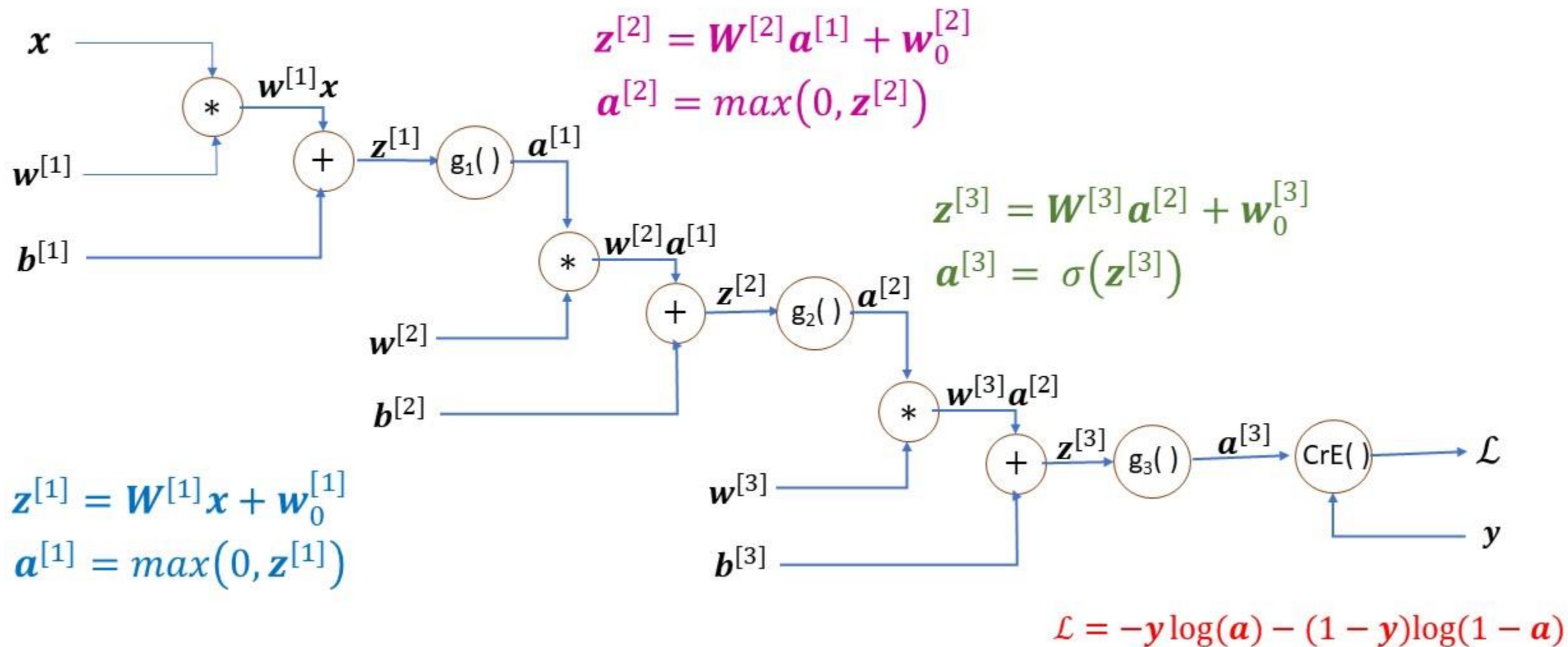


ML-ANN: Forward Pass

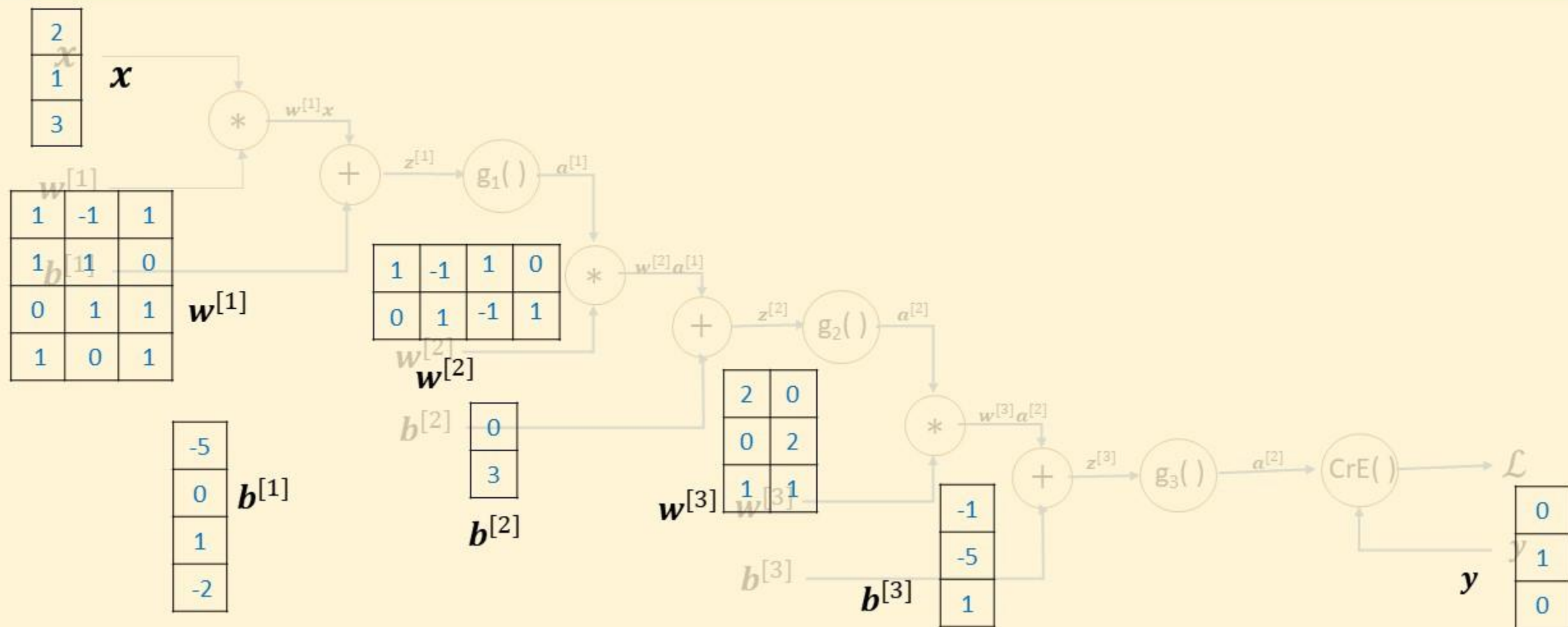


Loss $\mathcal{L}$ still a scalar!

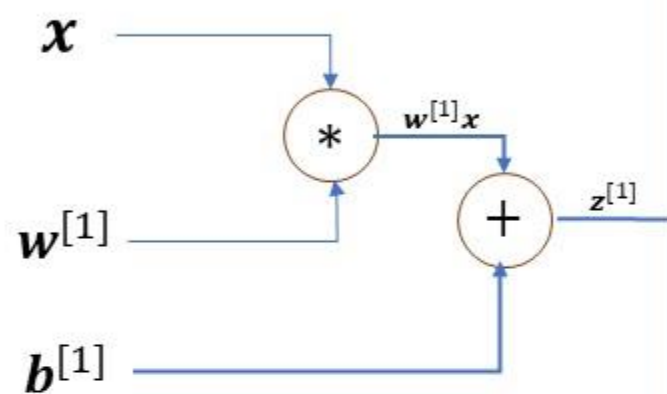
ML-ANN: Forward Pass



ML-ANN: Initialization



ML-ANN: Forward Pass



1	-1	1
1	1	0
0	1	1
1	0	1

2
1
3

 $+$

-5
0
1
-2

 $=$

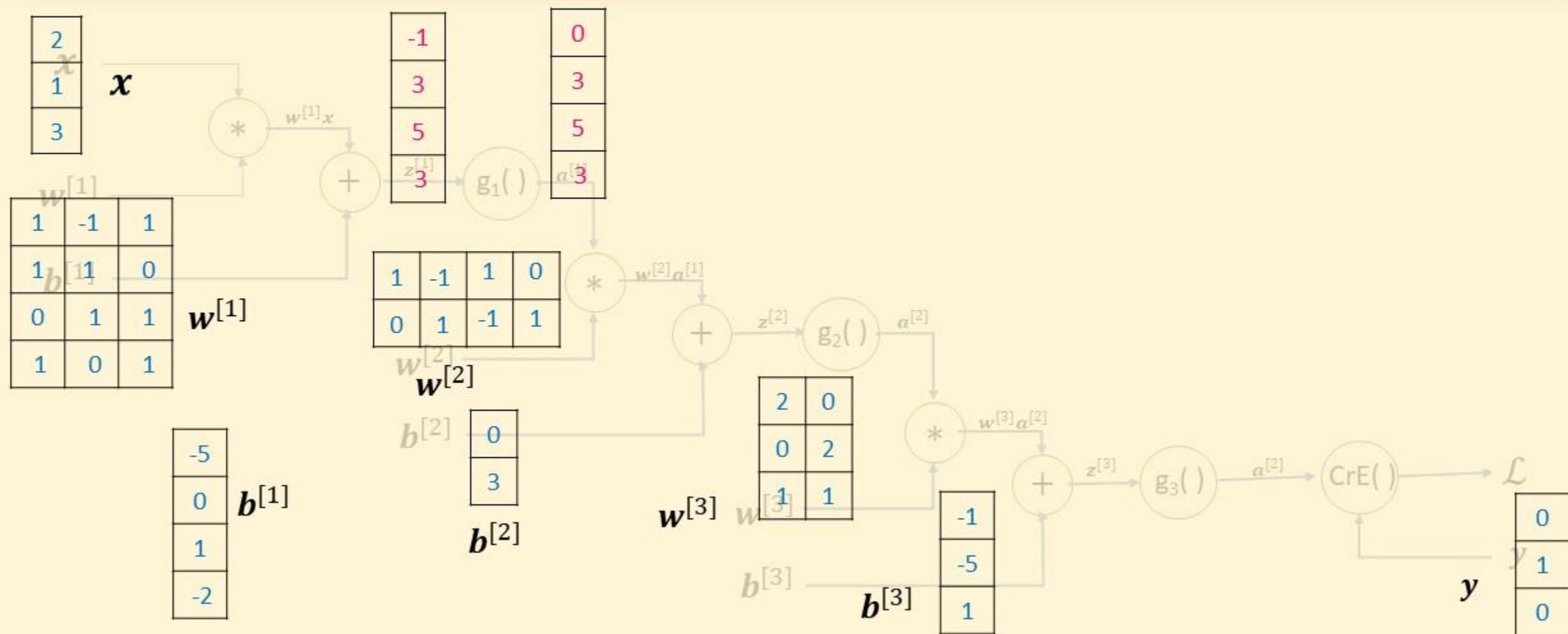
-1
3
5
3

$z^{[1]}$

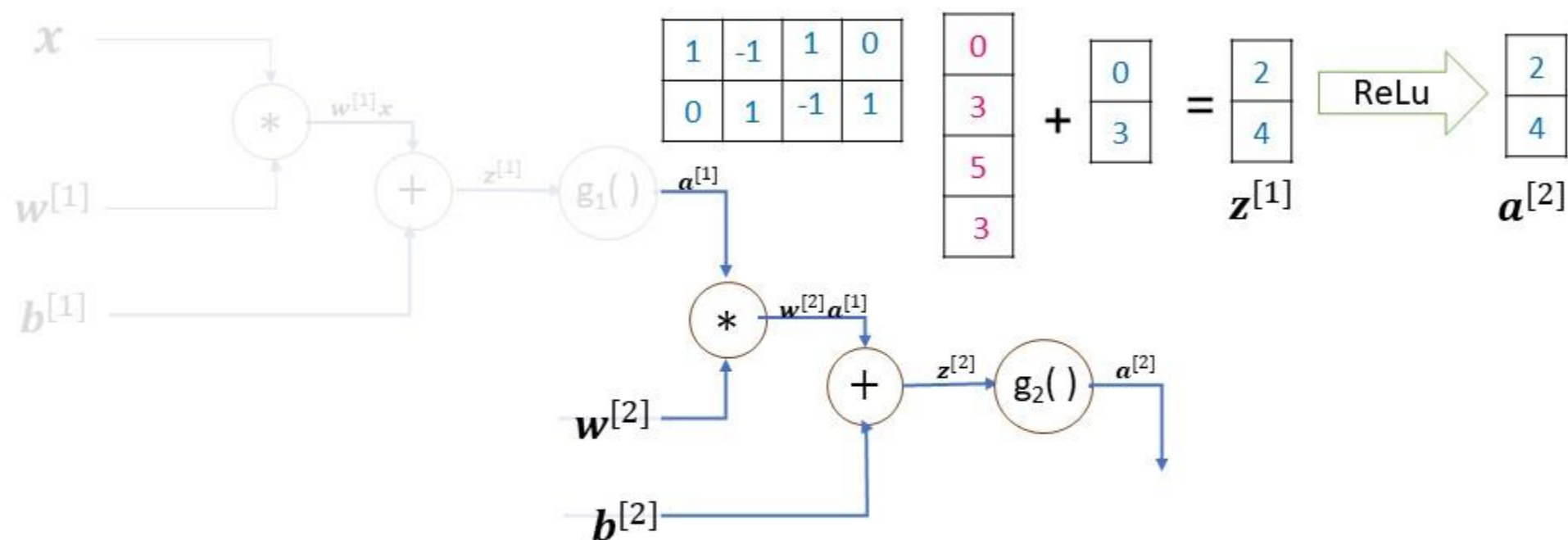
ReLU $\rightarrow$

0
3
5
3

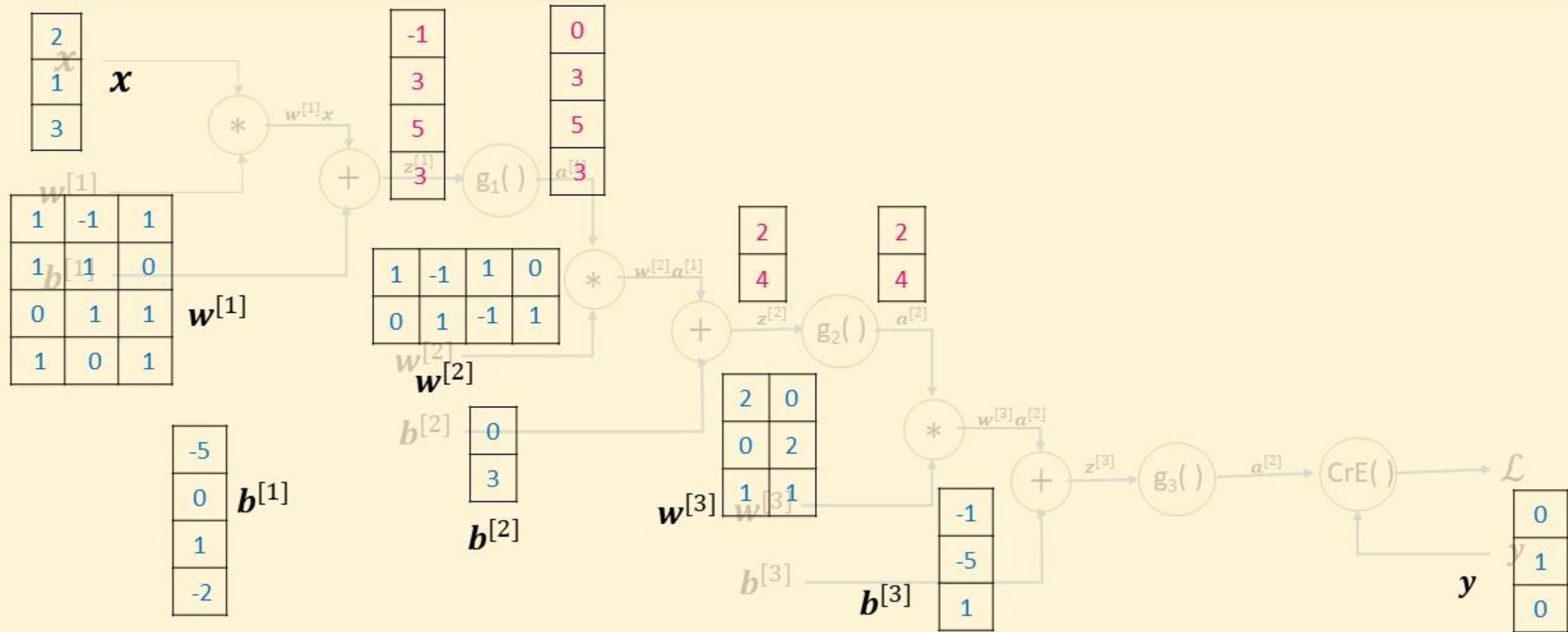
ML-ANN: Initialization



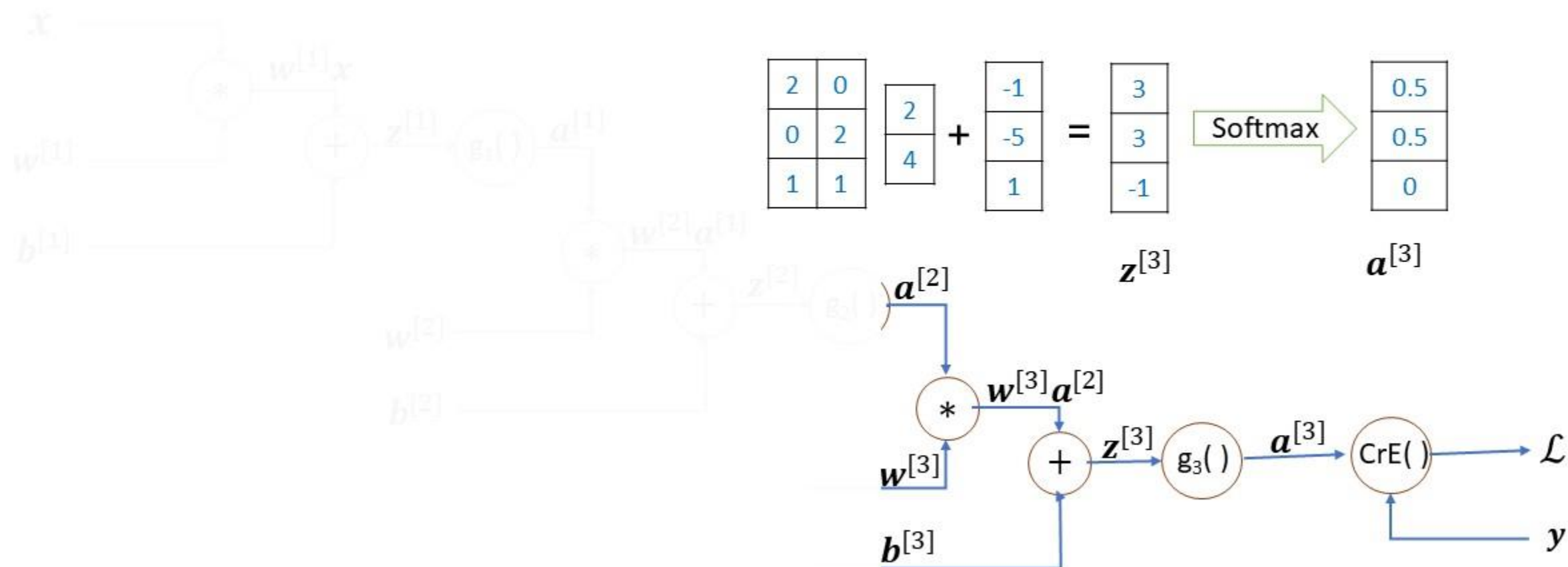
ML-ANN: Forward Pass



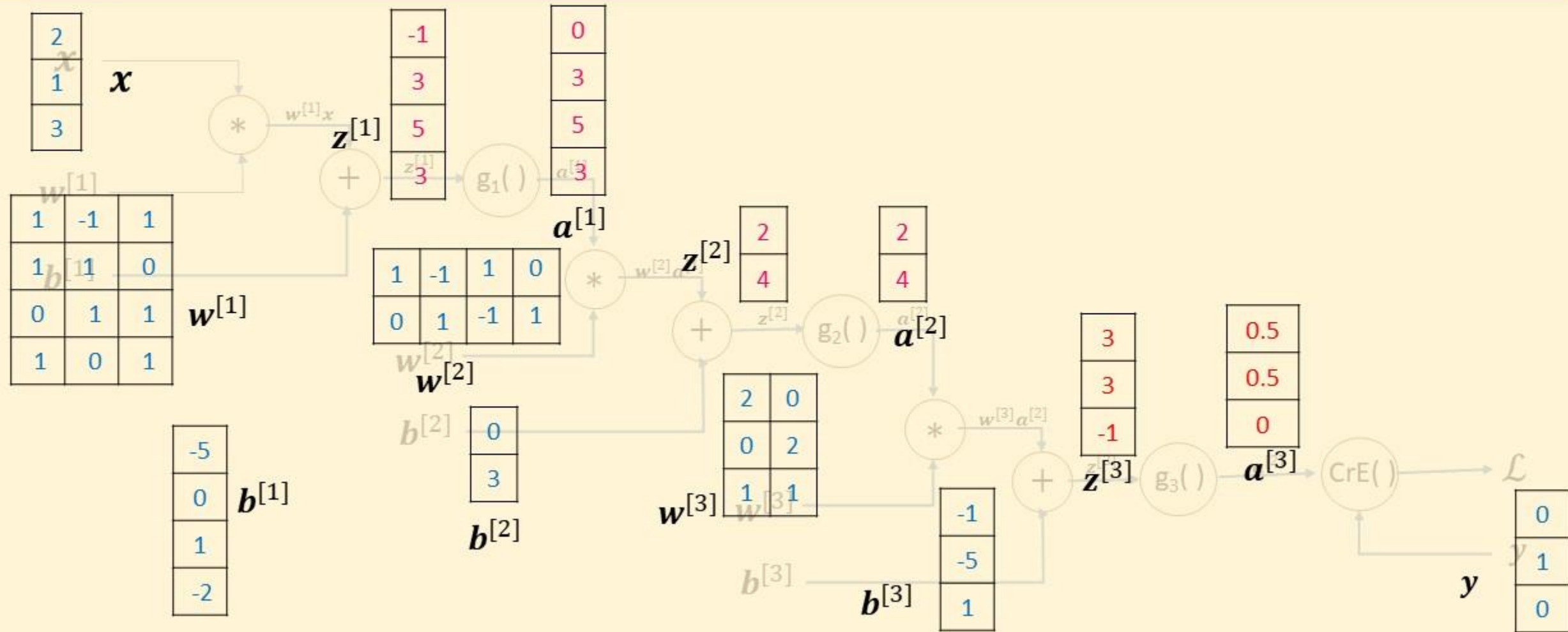
ML-ANN: Initialization



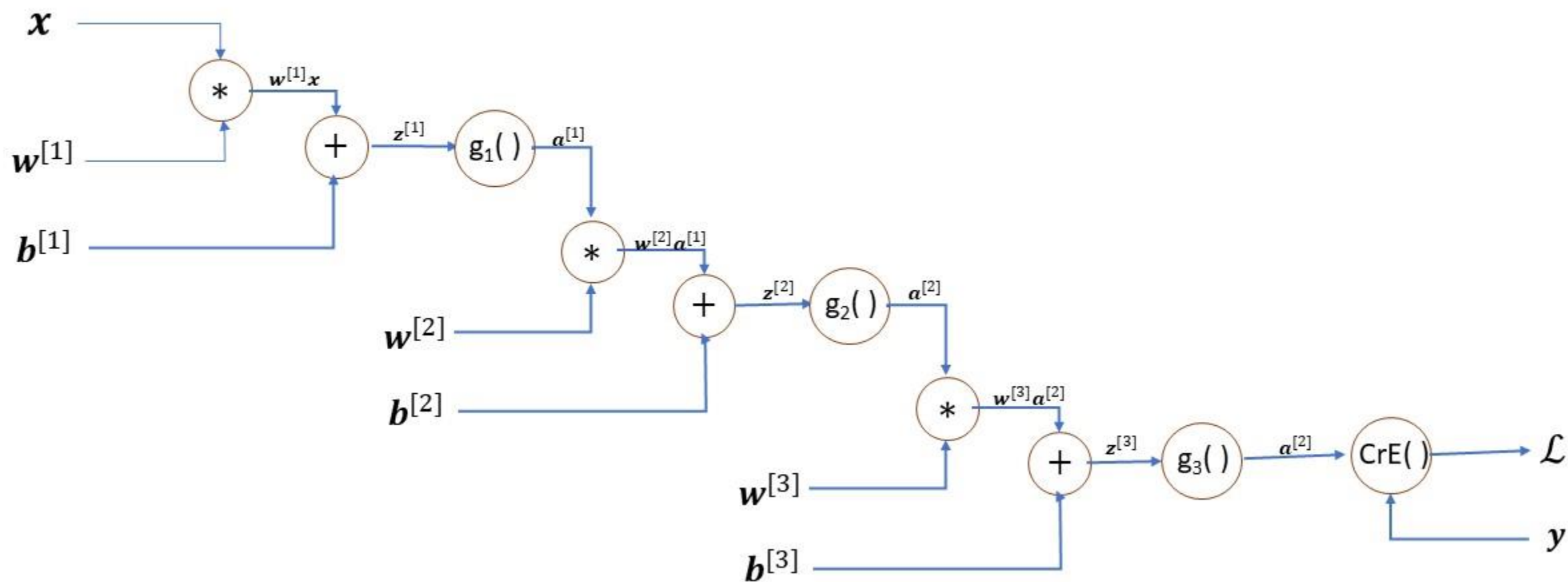
ML-ANN: Forward Pass



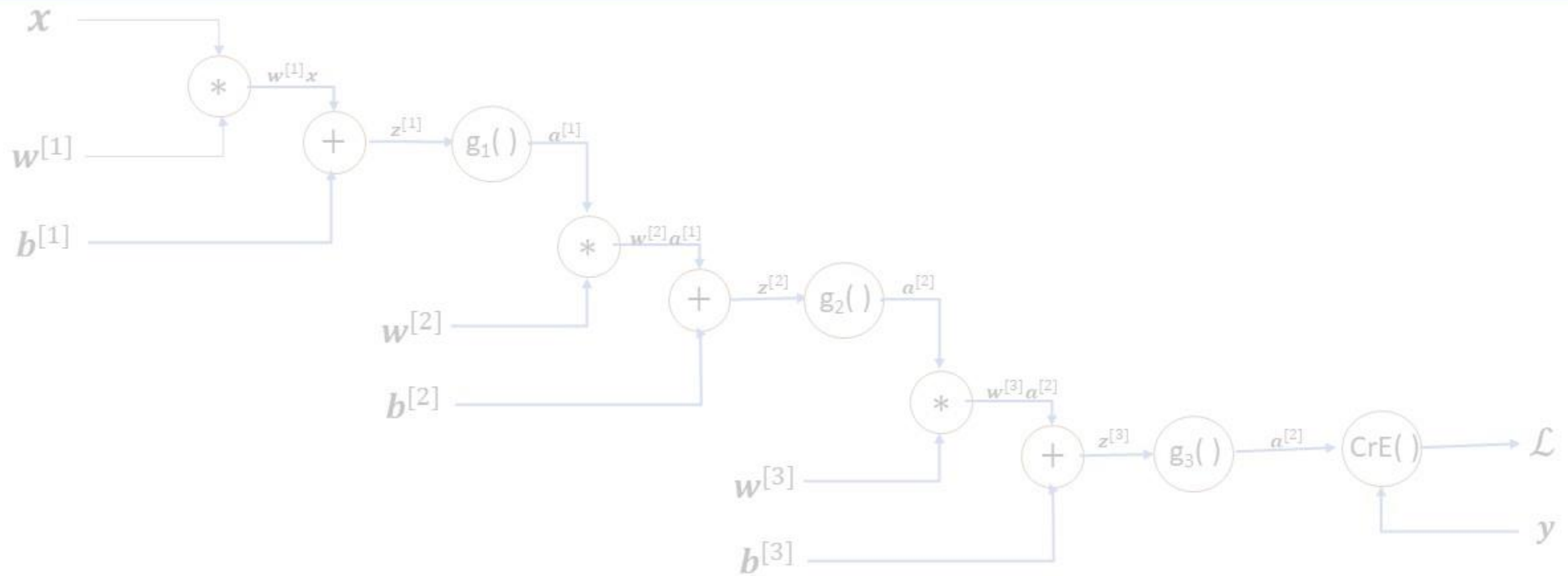
ML-ANN: Initialization



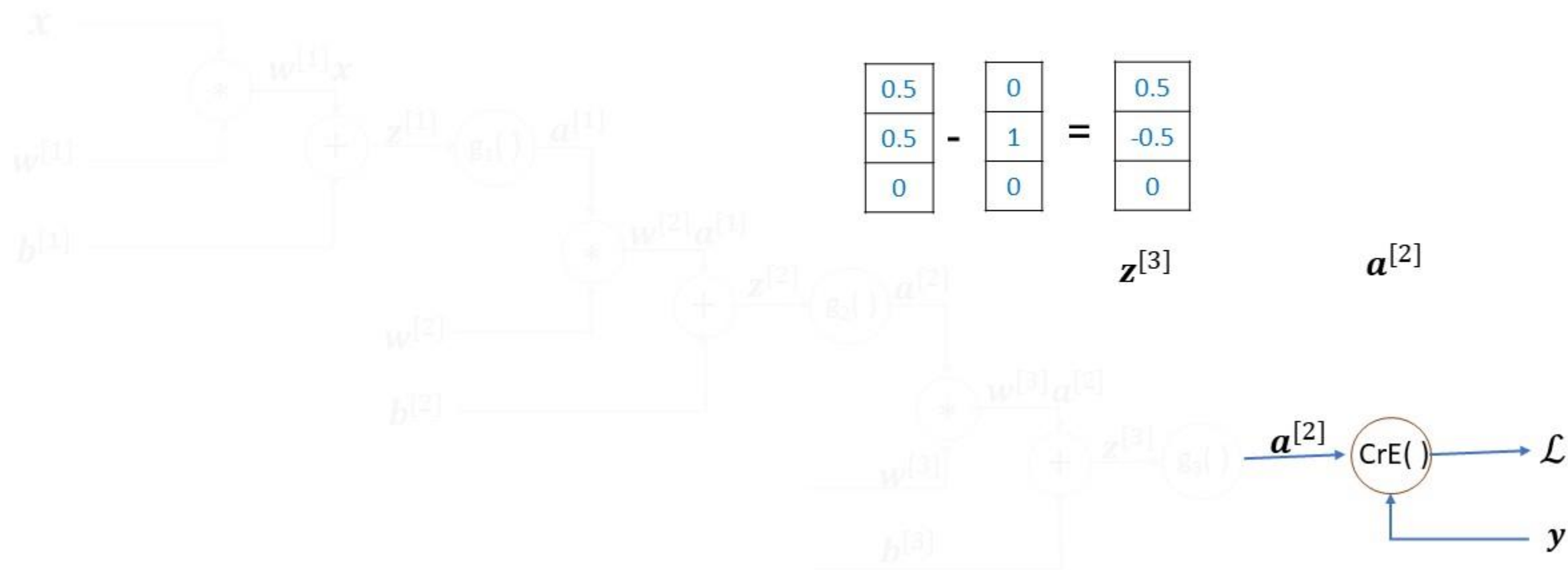
ML-ANN: Initialization



ML-ANN: Initialization

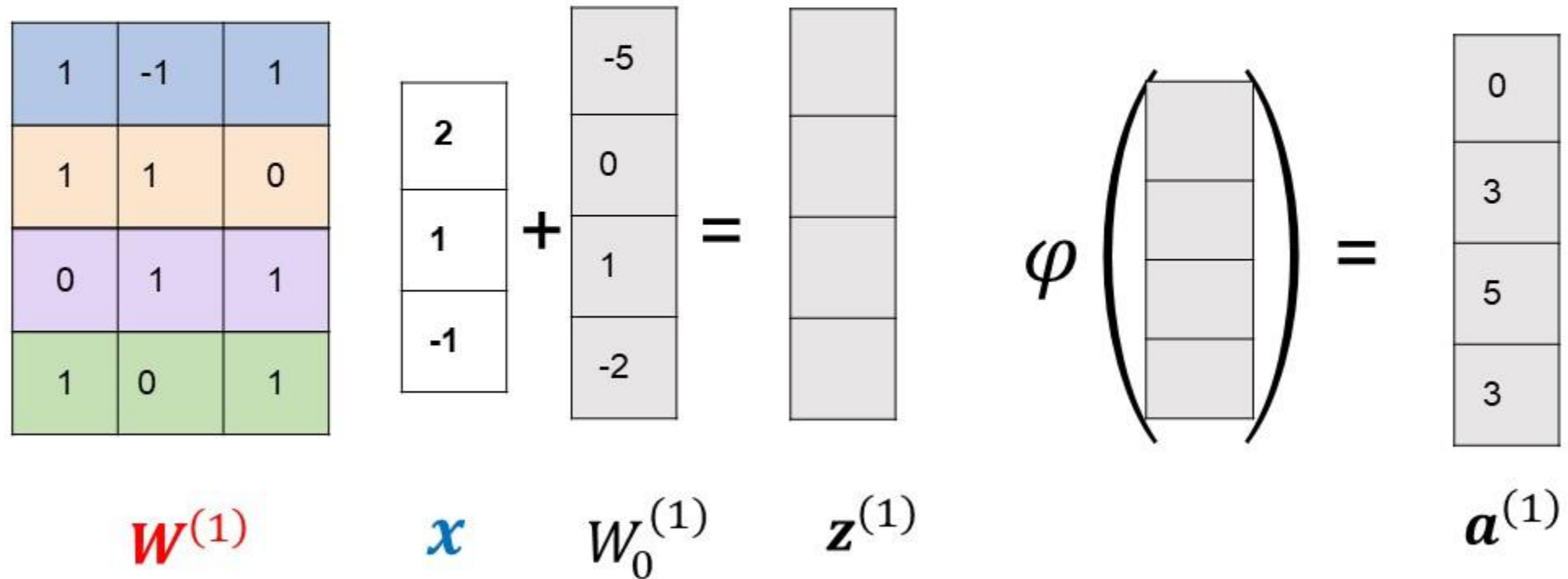


ML-ANN: Backward Pass



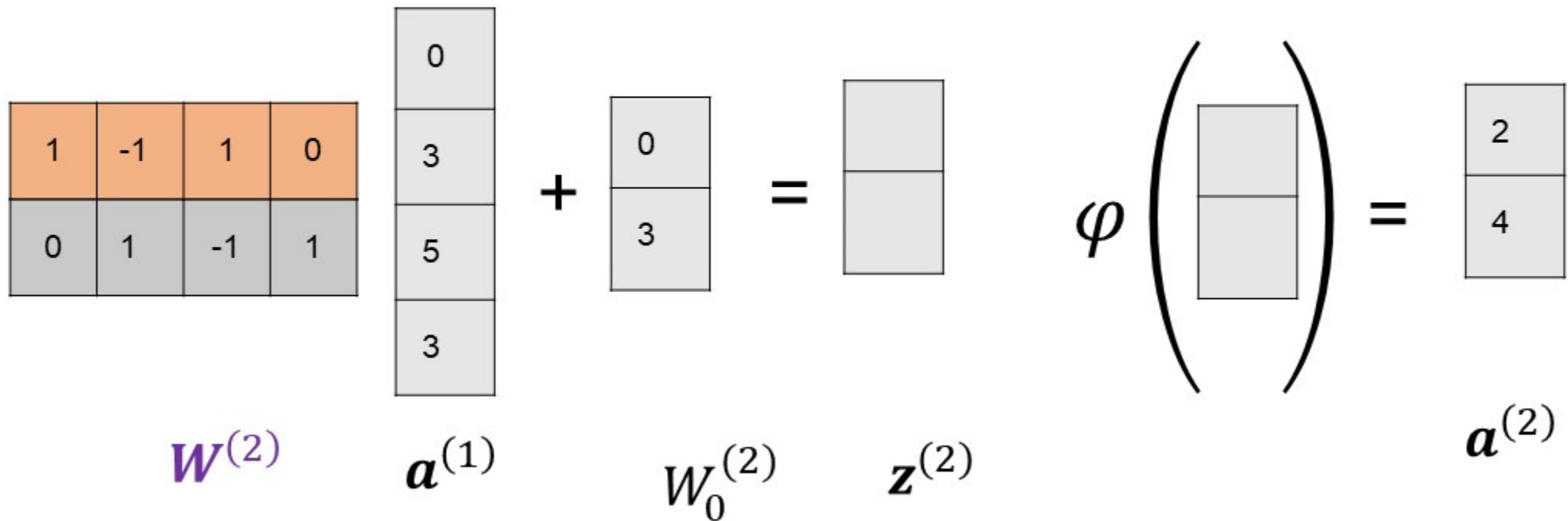
Backpropagation: Forward

Layer 1:



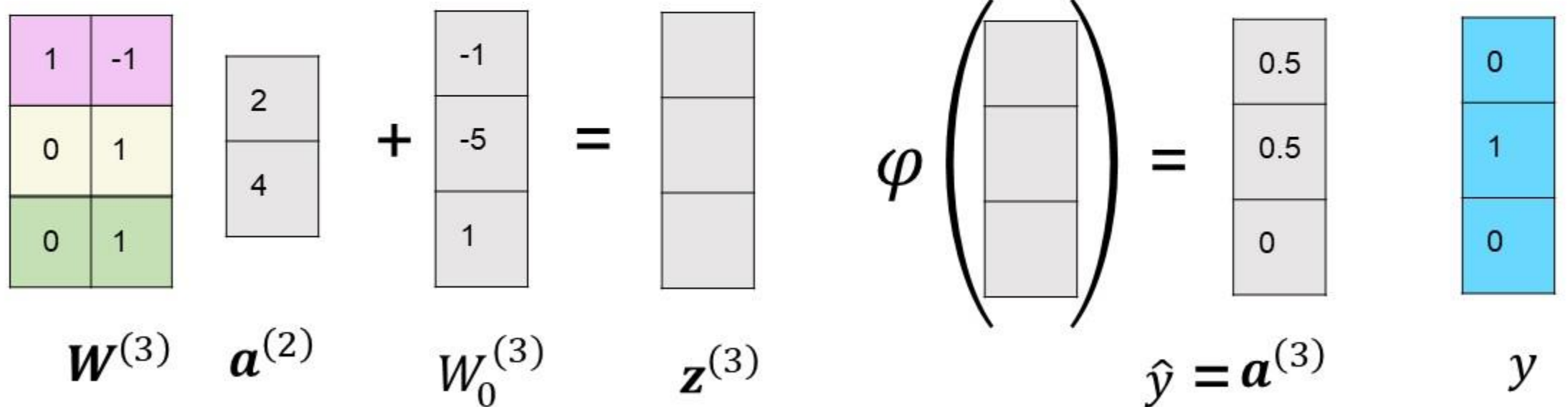
Backpropagation: Forward

Layer 2:



Backpropagation: Forward

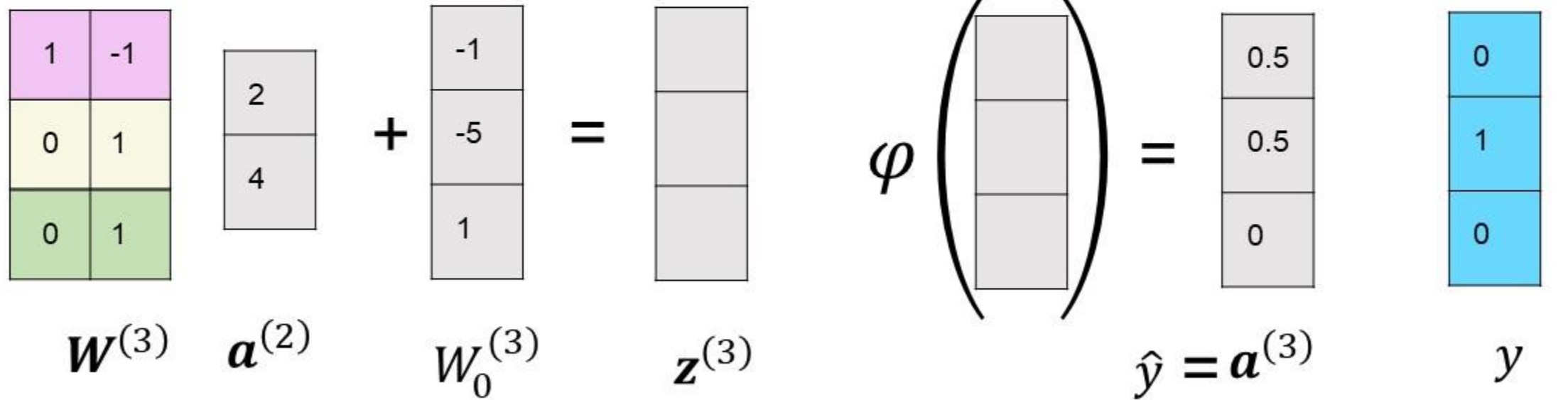
Layer 3:



L: Cross – Entropy Loss

Backpropagation: Backward

Layer 3:



L: Cross – Entropy Loss